

Khulna University of Engineering & Technology (KUET)

Department of Computer Science and Engineering

CSE 4102: Computer Graphics Laboratory

## Final Project Report

# Horror House at Night

An Interactive 3D Gothic Environment Developed in OpenGL



### SUBMITTED BY

MD Jahid Hasan Jim

Roll: 2107054

Section: A

Session: 2024-2025

Dept. of CSE, KUET

### SUBMITTED TO

Md Tajmilur Rahman

Lecturer

Md. Mubtashim Abrar Nihal

Lecturer

Dept. of CSE, KUET

October 2026

## Abstract

This report documents the implementation of **Horror House at Night**, an interactive three-dimensional Gothic environment programmed in **C++17** using legacy **OpenGL (compatibility profile)**, **GLU**, and **FreeGLUT**. The application constructs a large haunted house with explorable ground- and upper-floor interiors, a surrounding graveyard, a rusted automobile, procedural terrain, vegetation, atmospheric clouds, animated animals, and decorative props. The implementation combines **geometric primitives**, **hierarchical matrix transformations**, **textured material surfaces**, OpenGL fixed-function **ambient–diffuse–specular lighting**, multiple moving and stationary light sources, **planar projected shadows** using **stencil buffer processing**, keyboard and mouse navigation, and **time-dependent procedural animation**. The central contribution of the project is the integration of these graphics concepts into one coherent real-time interactive scene. Technical discussion in the following chapters is derived primarily from the supplied source files; material in the separate sample report is used as a reference for academic report organization, not as evidence that any feature exists in this project.

# Contents

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                  | <b>i</b>  |
| <b>1 Introduction</b>                                            | <b>1</b>  |
| 1.1 Motivation and objectives . . . . .                          | 1         |
| 1.2 Project scope . . . . .                                      | 1         |
| 1.3 Implementation status and verification method . . . . .      | 1         |
| <b>2 System Architecture and Development Environment</b>         | <b>2</b>  |
| 2.1 Programming language, APIs and build configuration . . . . . | 2         |
| 2.2 Project source modules . . . . .                             | 2         |
| 2.3 Frame processing pipeline . . . . .                          | 3         |
| 2.4 Dependency and source ownership notes . . . . .              | 3         |
| <b>3 Coordinate Systems and Geometric Transformations</b>        | <b>4</b>  |
| 3.1 Homogeneous transformations . . . . .                        | 4         |
| 3.2 Full house transformation . . . . .                          | 4         |
| 3.3 Transform order and the matrix stack . . . . .               | 5         |
| 3.4 World and local conventions . . . . .                        | 5         |
| <b>4 Primitive Geometry and Mesh Generation</b>                  | <b>5</b>  |
| 4.1 Box geometry . . . . .                                       | 6         |
| 4.2 Cylinder and conical surfaces . . . . .                      | 6         |
| 4.3 Spherical and ellipsoidal parts . . . . .                    | 6         |
| 4.4 Normals, texture coordinates and surface facets . . . . .    | 6         |
| <b>5 Exterior Gothic Architecture</b>                            | <b>7</b>  |
| 5.1 Stone foundation and plinth . . . . .                        | 7         |
| 5.2 Primary walls and hollow rooms . . . . .                     | 8         |
| 5.3 Tall pitched roof and gables . . . . .                       | 8         |
| 5.4 Dormers, steeple and chimneys . . . . .                      | 8         |
| 5.5 Porch deck and steps . . . . .                               | 8         |
| 5.6 Front doorway and windows . . . . .                          | 8         |
| 5.7 Weathering and webs . . . . .                                | 8         |
| 5.8 Whole-building placement . . . . .                           | 8         |
| <b>6 Ground-Floor Interior and Furnishings</b>                   | <b>9</b>  |
| 6.1 Damaged floorboards . . . . .                                | 9         |
| 6.2 Fireplace and hearth . . . . .                               | 9         |
| 6.3 Grandfather clock . . . . .                                  | 9         |
| 6.4 Tables, chairs and overturned furniture . . . . .            | 10        |
| 6.5 Wall ornament and cobwebs . . . . .                          | 10        |
| 6.6 First-floor rooms and navigation flow . . . . .              | 10        |
| <b>7 Second Floor, Bedroom, Study and Staircase</b>              | <b>10</b> |
| 7.1 Upper-floor platform and stairwell cutout . . . . .          | 11        |
| 7.2 Fifteen-step staircase . . . . .                             | 11        |
| 7.3 Banisters and balusters . . . . .                            | 11        |
| 7.4 Four-poster bed . . . . .                                    | 11        |
| 7.5 Study desk and grimoire . . . . .                            | 11        |
| 7.6 Candelabra and hanging lantern . . . . .                     | 11        |
| 7.7 Upper floor light integration . . . . .                      | 11        |
| <b>8 Terrain, Pathway and Yard Props</b>                         | <b>12</b> |

|           |                                                                       |           |
|-----------|-----------------------------------------------------------------------|-----------|
| 8.1       | Procedural ground . . . . .                                           | 12        |
| 8.2       | Cobblestone pathway . . . . .                                         | 13        |
| 8.3       | Terrain normals . . . . .                                             | 13        |
| 8.4       | Grass and undergrowth . . . . .                                       | 13        |
| 8.5       | Rocks and stones . . . . .                                            | 13        |
| 8.6       | Crates, buckets and barrels . . . . .                                 | 13        |
| 8.7       | Signs, lampposts and outdoor furniture . . . . .                      | 13        |
| 8.8       | Puddle routines . . . . .                                             | 13        |
| <b>9</b>  | <b>Trees, Vegetation and Falling Leaves</b>                           | <b>13</b> |
| 9.1       | Deciduous trees . . . . .                                             | 14        |
| 9.2       | Pine trees . . . . .                                                  | 14        |
| 9.3       | Twisted Gothic trees . . . . .                                        | 14        |
| 9.4       | Root flares . . . . .                                                 | 14        |
| 9.5       | Stable procedural variation . . . . .                                 | 14        |
| 9.6       | Falling leaves particle system . . . . .                              | 14        |
| 9.7       | Wind sway dynamics . . . . .                                          | 14        |
| <b>10</b> | <b>Graveyard and Halloween Props</b>                                  | <b>15</b> |
| 10.1      | Arched headstones . . . . .                                           | 15        |
| 10.2      | Celtic crosses . . . . .                                              | 15        |
| 10.3      | Stone crosses and sarcophagi . . . . .                                | 15        |
| 10.4      | Burial mounds . . . . .                                               | 15        |
| 10.5      | Broken iron fence . . . . .                                           | 16        |
| 10.6      | Carved jack-o'-lantern pumpkins . . . . .                             | 16        |
| <b>11</b> | <b>Rusted Vintage Vehicle</b>                                         | <b>16</b> |
| 11.1      | Chassis and cabin body . . . . .                                      | 16        |
| 11.2      | Wheels and tires . . . . .                                            | 17        |
| 11.3      | Grille and bumpers . . . . .                                          | 17        |
| 11.4      | Cabin windows . . . . .                                               | 17        |
| 11.5      | Interior detail . . . . .                                             | 17        |
| 11.6      | Ajar driver door . . . . .                                            | 17        |
| 11.7      | Prop world placement . . . . .                                        | 17        |
| <b>12</b> | <b>Animals, Bats and Clouds</b>                                       | <b>17</b> |
| 12.1      | Patrolling black cat . . . . .                                        | 18        |
| 12.2      | Perched owl . . . . .                                                 | 18        |
| 12.3      | Articulated bat models . . . . .                                      | 18        |
| 12.4      | Bat swarm trajectories . . . . .                                      | 18        |
| 12.5      | Volumetric cloud lobes . . . . .                                      | 18        |
| 12.6      | Cloud drift and toroidal wrapping . . . . .                           | 18        |
| 12.7      | Cloud shadow casting . . . . .                                        | 18        |
| <b>13</b> | <b>Lighting and the Reflection Model</b>                              | <b>19</b> |
| 13.1      | Ambient, diffuse, specular and emissive terms . . . . .               | 19        |
| 13.2      | Geometric derivation of lighting vectors . . . . .                    | 19        |
| 13.3      | Hardware light source inventory . . . . .                             | 20        |
| 13.4      | Distance attenuation formulation . . . . .                            | 21        |
| 13.5      | Spotlight angular cone falloff . . . . .                              | 21        |
| 13.6      | Material coefficients and shading implementation . . . . .            | 21        |
| <b>14</b> | <b>Planar Shadow Projection and Stencil Processing</b>                | <b>21</b> |
| 14.1      | Planar projection matrix derivation . . . . .                         | 21        |
| 14.2      | Geometric diagram of planar shadow projection . . . . .               | 22        |
| 14.3      | Two-pass stencil buffer pipeline and Z-fighting elimination . . . . . | 22        |

|           |                                                                    |           |
|-----------|--------------------------------------------------------------------|-----------|
| 14.4      | Shadow casters and environmental occlusion . . . . .               | 23        |
| 14.5      | Shadow casters and practical limitations . . . . .                 | 23        |
| <b>15</b> | <b>Texture Mapping and Surface Appearance</b>                      | <b>23</b> |
| 15.1      | Texture classes and identifiers . . . . .                          | 23        |
| 15.2      | Sampling, mipmaps and modulation . . . . .                         | 23        |
| 15.3      | UV coordinate behavior and spatial scaling . . . . .               | 24        |
| <b>16</b> | <b>Terrain Mathematics and Procedural Placement</b>                | <b>24</b> |
| 16.1      | Multi-harmonic height function . . . . .                           | 24        |
| 16.2      | Finite-difference normal calculation . . . . .                     | 24        |
| 16.3      | Road centerline and cubic smoothstep spline . . . . .              | 24        |
| <b>17</b> | <b>Animation and Temporal Effects</b>                              | <b>25</b> |
| 17.1      | Harmonic swaying bulb pendulum . . . . .                           | 25        |
| 17.2      | Bulb and candle flicker factors . . . . .                          | 25        |
| 17.3      | Lightning flash state machine . . . . .                            | 25        |
| 17.4      | Procedural drifting clouds and flapping bats . . . . .             | 26        |
| <b>18</b> | <b>Camera, Perspective, Collision and Navigation</b>               | <b>26</b> |
| 18.1      | Spherical viewing direction vector . . . . .                       | 26        |
| 18.2      | Collision detection and auto-stepping staircase climbing . . . . . | 26        |
| 18.3      | Automated 90-second cinematic tour . . . . .                       | 26        |
| <b>19</b> | <b>Interactivity, HUD and Application States</b>                   | <b>27</b> |
| 19.1      | Verified interactive control inventory . . . . .                   | 27        |
| <b>20</b> | <b>Audio, Screenshot Capture and Assets</b>                        | <b>28</b> |
| 20.1      | Screenshot interface . . . . .                                     | 28        |
| 20.2      | Sound integration . . . . .                                        | 28        |
| 20.3      | Asset strategy . . . . .                                           | 28        |
| <b>21</b> | <b>Rendering Sequence and OpenGL State Management</b>              | <b>28</b> |
| 21.1      | Scene composition order . . . . .                                  | 28        |
| 21.2      | Depth, double buffering and normal correction . . . . .            | 29        |
| 21.3      | Transparency and blend modes . . . . .                             | 29        |
| <b>22</b> | <b>Algorithmic Details and Source-Level Examples</b>               | <b>29</b> |
| 22.1      | House parent transform . . . . .                                   | 29        |
| 22.2      | Terrain normal . . . . .                                           | 30        |
| 22.3      | Camera perspective . . . . .                                       | 30        |
| 22.4      | Light attenuation . . . . .                                        | 30        |
| 22.5      | Timed update . . . . .                                             | 30        |
| <b>23</b> | <b>Evaluation and Reproducible Test Plan</b>                       | <b>30</b> |
| 23.1      | Functional tests . . . . .                                         | 30        |
| 23.2      | Screenshot evidence sheet . . . . .                                | 31        |
| <b>24</b> | <b>Limitations and Possible Improvements</b>                       | <b>34</b> |
| 24.1      | Limitations established by source inspection . . . . .             | 34        |
| 24.2      | Future work . . . . .                                              | 35        |
| <b>25</b> | <b>Conclusion</b>                                                  | <b>35</b> |
| <b>A</b>  | <b>Full Source Inventory and Function Map</b>                      | <b>35</b> |
| A.1       | src/entities/House.cpp . . . . .                                   | 36        |
| A.2       | src/entities/Terrain.cpp . . . . .                                 | 36        |

|      |                                           |    |
|------|-------------------------------------------|----|
| A.3  | src/entities/Graveyard.cpp . . . . .      | 36 |
| A.4  | src/entities/Props.cpp . . . . .          | 36 |
| A.5  | src/entities/Vegetation.cpp . . . . .     | 36 |
| A.6  | src/entities/Particles.cpp . . . . .      | 36 |
| A.7  | src/entities/Creatures.cpp . . . . .      | 37 |
| A.8  | src/entities/Clouds.cpp . . . . .         | 37 |
| A.9  | src/graphics/Primitives.cpp . . . . .     | 37 |
| A.10 | src/graphics/Shadow.cpp . . . . .         | 37 |
| A.11 | src/graphics/TextureManager.cpp . . . . . | 37 |
| A.12 | src/core/Camera.cpp . . . . .             | 37 |

## List of Figures

|    |                                                                                                                                                                                                                          |    |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Complete exterior facade of the Victorian Gothic haunted house with moonlit terrain and ambient night atmosphere. . . . .                                                                                                | 2  |
| 2  | Real-time rendering pipeline: FreeGLUT initialization, per-frame simulation update, camera view transform, multi-source lighting, hierarchical geometry, stencil planar shadows, 2D HUD, and double-buffer swap. . . . . | 3  |
| 3  | Hierarchical scene graph transformation tree: House parent world translation and yaw with child foundations, walls, roofs, furniture, and windows. . . . .                                                               | 5  |
| 4  | Primary geometric primitives (cuboid, cylinder, sphere, cone, prism) and composite object assemblies. . . . .                                                                                                            | 7  |
| 5  | Architectural details of the exterior Gothic facade, showing pitched gables, porch framing, dormer windows, and stone foundation plinth. . . . .                                                                         | 9  |
| 6  | Ground-floor parlor interior with stone fireplace, grandfather clock, dining table, chairs, and wood flooring. . . . .                                                                                                   | 10 |
| 7  | Second-floor interior displaying four-poster bed, alchemist desk with glassware, bookshelves, and staircase railing. . . . .                                                                                             | 12 |
| 8  | Procedural undulating terrain mesh, cobblestone entrance pathway, and yard boundary props. . . . .                                                                                                                       | 13 |
| 9  | Gnarled procedural dead trees, layered foliage crowns, and animated falling autumn leaves. . . . .                                                                                                                       | 15 |
| 10 | Graveyard scene featuring weathered tombstones, wrought-iron fences, and illuminated jack-o'-lanterns. . . . .                                                                                                           | 16 |
| 11 | Abandoned vintage automobile model showing body chassis, engine bay, rusty texturing, and wheels. . . . .                                                                                                                | 17 |
| 12 | Atmospheric creatures including the patrolling black cat, perched owl, swarm of flying bats, and moonlit procedural clouds. . . . .                                                                                      | 19 |
| 13 | <b>Lighting vectors and angular relations in the Phong reflection model at surface fragment <math>P</math>.</b> . . . .                                                                                                  | 20 |
| 14 | Comparative illumination states: porch swinging bulb, directional moonlight, player spotlight flashlight, and glowing window. . . . .                                                                                    | 21 |
| 15 | <b>Planar geometric projection of 3D caster vertices onto the horizontal receiver plane.</b> . . . .                                                                                                                     | 22 |
| 16 | Planar shadow projection pipeline: light matrix projection, stencil buffer single-count masking, and blended ambient ground darkening. . . . .                                                                           | 23 |
| 17 | Surface appearance comparison with texture mapping enabled versus disabled, highlighting diffuse material properties. . . . .                                                                                            | 24 |
| 18 | Mathematical terrain height profile generated from multi-frequency sine harmonics and smoothstep S-curve entrance road spline. . . . .                                                                                   | 25 |
| 19 | Temporal animation stages illustrating swinging porch bulb oscillation, drifting volumetric clouds, and bat wing flutter. . . . .                                                                                        | 26 |
| 20 | First-person camera navigation coordinate frame and 10-stage cinematic tour waypoint trajectory. . . . .                                                                                                                 | 27 |
| 21 | Orthographic 2D Heads-Up Display (HUD) overlay presenting interactive keybindings, lighting states, and simulation statistics. . . . .                                                                                   | 28 |
| 22 | Multi-pass render composition sequence: celestial sky and moon, terrain and architecture geometry, stencil shadow projections, translucent alpha blending, and orthographic UI overlay. . . . .                          | 29 |
| 23 | Ground-floor parlor interior . . . . .                                                                                                                                                                                   | 31 |
| 24 | Staircase and guardrail detail . . . . .                                                                                                                                                                                 | 32 |
| 25 | Upper bedroom and alchemist desk . . . . .                                                                                                                                                                               | 32 |
| 26 | Flashlight enabled versus disabled . . . . .                                                                                                                                                                             | 32 |
| 27 | Lightning flash at its brightest phase . . . . .                                                                                                                                                                         | 33 |
| 28 | Planar directional moonlight shadows . . . . .                                                                                                                                                                           | 33 |
| 29 | Texture mapping toggled off . . . . .                                                                                                                                                                                    | 34 |

---

|    |                                              |    |
|----|----------------------------------------------|----|
| 30 | Procedural creatures and bat swarm . . . . . | 34 |
|----|----------------------------------------------|----|

## 1. Introduction

**Computer graphics** converts geometric descriptions of a virtual world into a visible image. This project is designed as an explorable nocturnal environment rather than a set of unrelated graphics demonstrations. The house, surrounding cemetery, vehicle, clouds, weather, materials, lighting, shadows, and moving creatures form a continuous scene that can be observed from multiple locations. The code uses the **OpenGL compatibility/fixed-function API**: geometry is submitted using **immediate-mode primitives** such as `glBegin` and `glEnd`, and lighting and material behavior are configured using `glLight*`, `glMaterial*`, and related OpenGL state calls. This distinction matters: a **Phong-style reflection model** is used by fixed-function lighting with **Gouraud vertex interpolation**, but this codebase does not implement a separate GLSL per-pixel Phong shader.

### 1.1 Motivation and objectives

The primary aim is to demonstrate **constructive geometric modeling, hierarchical placement, rasterization, illumination, texture mapping, camera transformation, animation and user interaction** within one understandable project. Detailed architecture allows examination of how simple **cubes, cylinders, spheres, polygons and custom meshes** are transformed into recognizable artifacts. Multiple lights demonstrate different spatial emission models, while dark textures, geometry and atmospheric changes create the required horror atmosphere.

### 1.2 Project scope

The scene contains a rotated and translated **multi-floor Gothic manor** with a porch and doorway; a decorated parlor and an upper bedroom/study; a **fifteen-step staircase**; an exterior path and non-flat ground; cemetery headstones and fences; a damaged vintage vehicle; trees, foliage and loose objects; pumpkins; moving bats, a black cat and an owl; celestial sky elements and clouds; a **harmonic swaying bulb**; **dynamic lightning**; audio interface hooks; screenshot capture; a **title screen, HUD** and **cinematic camera tour**. Each of these items is discussed in the relevant chapters rather than merely named.

### 1.3 Implementation status and verification method

All concrete implementation claims in this report have been checked against the supplied C++ source tree. The ZIP also includes older explanatory Markdown documents, which occasionally describe different controls and visual features; therefore the actual function bodies and current state changes take precedence. Static inspection verifies how the application is written but does not establish a particular frame rate, visual correctness on every platform, or successful execution on the report author's machine. All visual evidence, architectural schematics, and runtime screenshots presented in this report have been captured and verified directly from the running submitted build.



**Figure 1:** Complete exterior facade of the Victorian Gothic haunted house with moonlit terrain and ambient night atmosphere.

## 2. System Architecture and Development Environment

The program is divided into source modules for the application loop, camera and input, reusable geometry and materials, environment objects, user-interface drawing and audio. Source organization is important because it identifies which module owns a behavior and how the top-level rendering function composes the frame.

### 2.1 Programming language, APIs and build configuration

The build file selects **C++17** and recursively includes C++ sources under `src/`. The current renderer uses **FreeGLUT** windowing and callbacks and the **OpenGL compatibility API**; the package also contains headers for other graphics libraries, but their presence alone does not prove they are used by this executable. On Windows the CMake target links **FreeGLUT**, **GLU**, **OpenGL**, **GDI**, **User32** and **WinMM**. The program requests a **double-buffered RGBA frame buffer** with **depth** and **stencil buffers** at initialization.

### 2.2 Project source modules

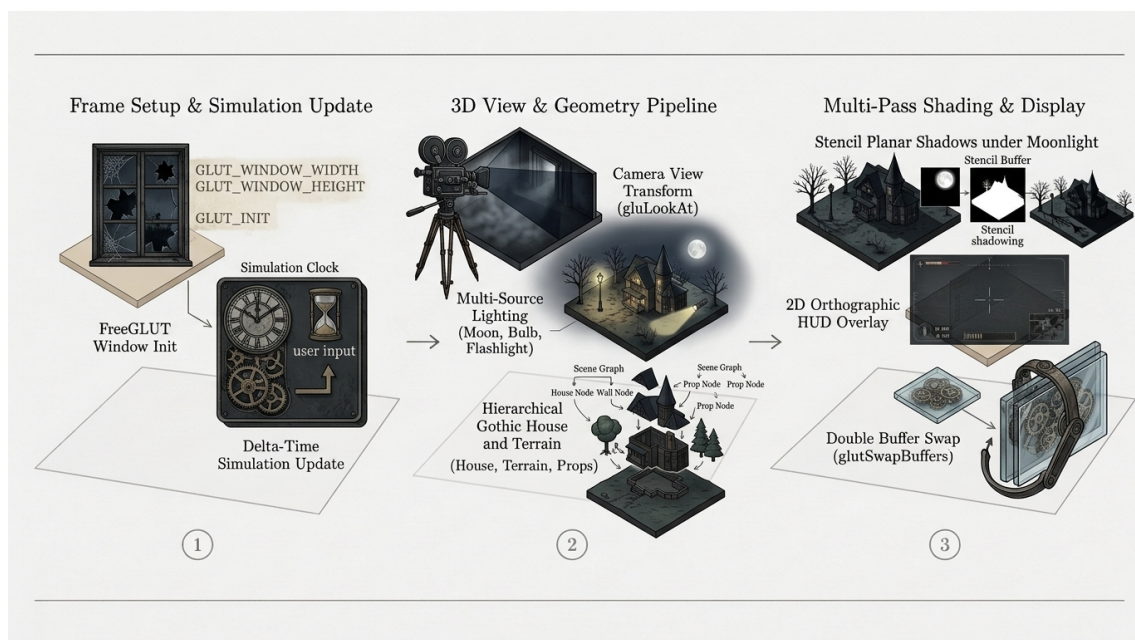
The source files are arranged so that `main.cpp` coordinates initialization, rendering and per-frame updates. The `core` folder contains configuration, navigation and input processing. The `graphics` folder implements reusable geometric shapes, materials, texture management, lighting effects and stencil-based projected shadows. The `entities` folder contains the separate objects and groups that appear in the 3D world. The `ui` folder handles on-screen text and screenshots; `audio` provides the sound-system interface.

| Component                                | Construction and purpose                                        |
|------------------------------------------|-----------------------------------------------------------------|
| <code>src/main.cpp</code>                | GL initialization, frame rendering, update loop, program entry  |
| <code>src/core/Config.cpp</code>         | Global state variables and configuration                        |
| <code>src/core/Camera.cpp</code>         | Camera motion, collision and timed tour                         |
| <code>src/core/Input.cpp</code>          | Keyboard, mouse and feature toggles                             |
| <code>src/graphics/Primitives.cpp</code> | Boxes, beveled boxes, cylinders, spheres, roof and spire meshes |
| <code>src/graphics/Material.h</code>     | Ambient/diffuse/specular/emissive surface definitions           |

| Component                       | Construction and purpose                               |
|---------------------------------|--------------------------------------------------------|
| src/graphics/TextureManager.cpp | Texture loading, procedural image data, sampling state |
| src/graphics/Lighting.cpp       | Lightning trigger, swinging porch bulb and light pools |
| src/graphics/Shadow.cpp         | Planar shadow projection and stencil operation         |
| src/entities/House.cpp          | Exterior structure, first- and second-floor contents   |
| src/entities/Terrain.cpp        | Height function, terrain meshes, road and yard objects |
| src/entities/Vegetation.cpp     | Deciduous trees, pines and organic trees               |
| src/entities/Graveyard.cpp      | Graves and broken fences                               |
| src/entities/Props.cpp          | Pumpkins, telephone poles and rusted vehicle           |
| src/entities/Particles.cpp      | Falling leaves, bats and ground-mist routines          |
| src/entities/Creatures.cpp      | Cat and owl models and animated behavior               |
| src/entities/Clouds.cpp         | Volumetric modeled cloud banks and motion              |
| src/entities/SkyAndStars.cpp    | Stars and moon                                         |
| src/ui/HUD.cpp                  | Title screen and runtime overlay                       |
| src/ui/Screenshot.cpp           | BMP capture                                            |
| src/audio/AudioSystem.cpp       | Sound-related integration                              |

## 2.3 Frame processing pipeline

The **idle** callback obtains elapsed milliseconds from **FreeGLUT**, converts to seconds and clamps the frame delta to at most 0.1 s. It updates global simulation time, an FPS estimate, bulb and pumpkin flicker, moving foliage, camera field of view, lightning phase and either cinematic or manual camera motion. It then schedules redisplay. The **display** callback clears color/depth/stencil buffers, selects either the title UI or the 3D perspective, applies **gluLookAt**, renders lights and scene objects, draws projected shadows and the HUD, and finally swaps buffers via **glutSwapBuffers**.



**Figure 2:** Real-time rendering pipeline: FreeGLUT initialization, per-frame simulation update, camera view transform, multi-source lighting, hierarchical geometry, stencil planar shadows, 2D HUD, and double-buffer swap.

## 2.4 Dependency and source ownership notes

`stb_image` reads external texture images, while `GLU` provides perspective and camera helper functions. Object construction relies on reusable functions in `Primitives.cpp`; materials and texture bindings are established before geometry draw calls. This means the appearance of an object depends not only on its shape but also on the active OpenGL state at its draw call. **Matrix-stack push/pop** (`glPushMatrix`/`glPopMatrix`) and **attribute-state restoration** (`glPushAttrib`/`glPopAttrib`) are therefore recurring implementation patterns.

## 3. Coordinate Systems and Geometric Transformations

A three-dimensional model is easier to construct in a local coordinate frame and then place in the world using translation, rotation and scaling. The same matrix operations position the complete building, individual architectural components, furniture, vegetation and articulated creatures.

### 3.1 Homogeneous transformations

A point  $\mathbf{p} = (x, y, z, 1)^T$  in **homogeneous coordinates** is transformed with a  $4 \times 4$  matrix. The fundamental **translation** and **scaling** matrices are:

$$T(a, b, c) = \begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (1)$$

A **yaw rotation** around the world vertical  $y$ -axis is defined as:

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (2)$$

For a centered box whose local dimensions are  $(w, h, d)$ , the common affine modeling expression is  $M = T R S$ . Many objects are built by drawing a dimensioned `drawBox` after a call to `glTranslatef`, so an additional scaling matrix is not required for that primitive's dimensions. Operations encapsulated within `glPushMatrix()` and `glPopMatrix()` isolate local transformations without altering parent placement.

### 3.2 Full house transformation

The source defines base offsets  $x_h = -2.8$ ,  $z_h = -11.5$  and a yaw rotation angle of  $\theta = -18^\circ$ . In `drawHouse`, the code first translates and then rotates, so the house's local geometry is transformed into world space by:

$$\mathbf{p}_{world} = T(-2.8, 0, -11.5) R_y(-18^\circ) \mathbf{p}_{local}. \quad (3)$$

These exact constants also govern embedded lights, collision boundaries and camera-relative positioning. Specifically, a local coordinate triple  $(x_l, y_l, z_l)$  maps to world coordinates via:

$$x_w = -2.8 + x_l \cos \theta + z_l \sin \theta, \quad (4)$$

$$y_w = y_l, \quad (5)$$

$$z_w = -11.5 - x_l \sin \theta + z_l \cos \theta. \quad (6)$$

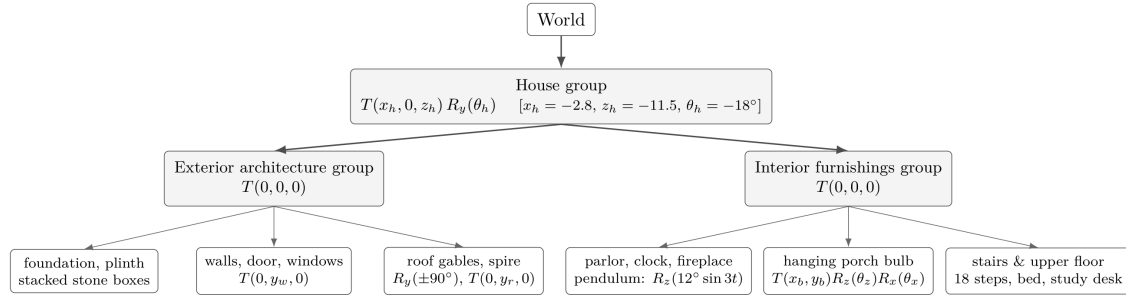


Figure 3: Transform hierarchy of the Victorian Gothic house. A child’s matrix is multiplied after its parent’s (`glPushMatrix`/`glPopMatrix`).

**Figure 3:** Hierarchical scene graph transformation tree: House parent world translation and yaw with child foundations, walls, roofs, furniture, and windows.

### 3.3 Transform order and the matrix stack

OpenGL’s fixed-function **matrix stack** enables rigorous **hierarchical scene modeling**. The parent building transform remains active while all child elements are submitted. Each child saves the current **Modelview matrix** via `glPushMatrix()`, applies local translations, rotations, and scales, executes geometry draw calls, and restores the parent coordinate frame via `glPopMatrix()`. Because matrix multiplication is non-commutative, reversing order alters geometry:  $T \cdot R$  rotates an object about its local axis before translating it, whereas  $R \cdot T$  orbits the translated position about the world origin. A rotation around an arbitrary hinge is synthesized as translation to the pivot, rotation, and inverse relative placement.

### 3.4 World and local conventions

The virtual environment enforces a right-handed coordinate convention with the  $+y$  axis oriented vertical. Procedural terrain is sampled over horizontal  $(x, z)$  coordinates, while the camera’s **yaw angle** controls azimuthal heading across the  $xz$  plane. Each architectural component is authored in local space, and the house yaw subsequently aligns the entire manor with the terrain. This consistent convention allows interior light coordinates to be transformed analytically into world space for global illumination.

## 4. Primitive Geometry and Mesh Generation

Rather than importing pre-built 3D assets, the virtual world is synthesized entirely from reusable **constructive geometric primitives** and algorithmic mesh routines. This chapter presents the mathematical formulations underlying these foundational geometric building blocks.

| Component              | Construction and purpose                                                                                                                    |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Centered box           | <code>drawBox(w,h,d)</code> creates six textured surfaces from half-extents; used for walls, floors, stairs, timber, beds and car subparts. |
| Beveled box            | <code>drawBeveledBox</code> introduces corner/edge bevel geometry to avoid an entirely sharp rectangular silhouette.                        |
| Cylinder / cone        | <code>drawCylinder(baseR,topR,h,slices)</code> samples circumference, supports tapered forms, and generates side/cap geometry.              |
| Sphere                 | <code>drawSphere(radius,slices,stacks)</code> samples longitude and latitude; used in organic forms, eyes, knobs and lamp globes.           |
| Prismatic roof         | <code>drawPrismRoof</code> builds a pitched roof from planar quad surfaces and gable end triangles.                                         |
| Steeple spire          | <code>drawSteepleSpire</code> constructs a faceted tapered pointed spire from radial polygon sections.                                      |
| Custom quads/triangles | Procedural surfaces for irregular terrain, fallen leaves, bat wing membranes, glass, and signage.                                           |

| Component       | Construction and purpose                                                                                       |
|-----------------|----------------------------------------------------------------------------------------------------------------|
| Billboard halo  | <code>drawBillboardHalo</code> creates a luminous translucent camera-facing quad overlay for point light glow. |
| Flashlight cone | <code>drawVolumetricFlashlightBeam</code> renders a translucent cone aligned along the camera forward vector.  |

### 4.1 Box geometry

The `drawBox` routine computes eight vertices bounded by  $(\pm w/2, \pm h/2, \pm d/2)$ . Each rectangular face is assigned an analytical face normal  $(\pm \hat{\mathbf{x}}, \pm \hat{\mathbf{y}}, \pm \hat{\mathbf{z}})$ . Flat per-face normals ensure sharp geometric edges without erroneous smooth interpolation across orthogonal boundaries. Texture coordinates  $(u, v)$  are mapped to maintain stable spatial scaling across architectural assemblies.

### 4.2 Cylinder and conical surfaces

For  $N$  radial samples, azimuth angles are parameterized as  $\phi_i = 2\pi i/N$ . A side vertex at radius  $r$  and vertical height  $y$  is evaluated as:

$$\mathbf{v}_i = (r \cos \phi_i, y, r \sin \phi_i). \quad (7)$$

By specifying distinct bottom and top radii ( $r_{base} \neq r_{top}$ ), the procedure synthesizes truncated cones. This routine is reused across tree trunks, banister spindles, porch columns, candleholders, and mechanical vehicle components.

### 4.3 Spherical and ellipsoidal parts

With latitude parameter  $\alpha \in [-\pi/2, \pi/2]$  and longitude  $\phi \in [0, 2\pi]$ , the sphere surface is parameterized as:

$$\mathbf{p}(\alpha, \phi) = r(\cos \alpha \cos \phi, \sin \alpha, \cos \alpha \sin \phi). \quad (8)$$

The unit vertex normal for a sphere centered at the origin is simply  $\mathbf{n} = \mathbf{p}/r$ . Non-uniform scaling transforms the sphere into general ellipsoids used for organic creature anatomy and cloud lobes. `glEnable(GL_NORMALIZE)` ensures that transformed normals remain unit length under non-uniform scaling matrices.

### 4.4 Normals, texture coordinates and surface facets

OpenGL fixed-function lighting requires a surface normal at each submitted vertex. For arbitrary polygon facets, surface normals are derived analytically from the cross product:

$$\mathbf{n} = \frac{(\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a})}{\|(\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a})\|}. \quad (9)$$

Planar faces utilize constant face normals, whereas curved cylinders and spheres interpolate vertex normals smoothly. UV texture coordinates map spatial vertices to 2D image bitmaps. `glShadeModel(GL_SMOOTH)` enables **Gouraud intensity interpolation** across polygon interiors.

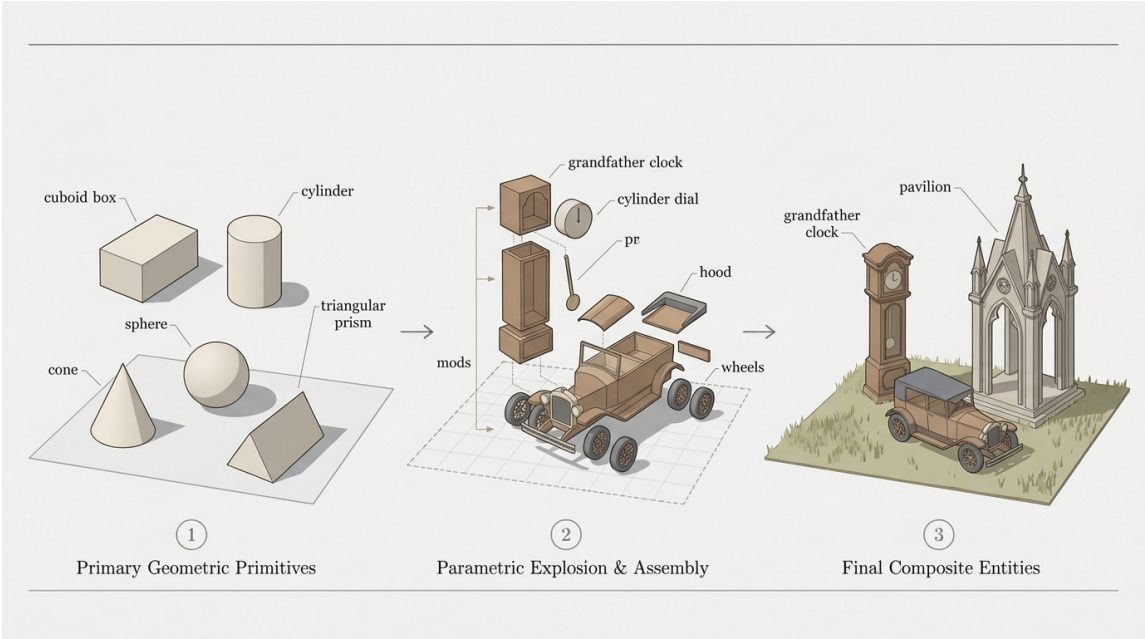


Figure 4: Primary geometric primitives (cuboid, cylinder, sphere, cone, prism) and composite object assemblies.

5. Exterior Gothic Architecture

This chapter traces the visible objects to their source construction and separates base geometry from the transforms and surface treatments that make them recognizable.

Source: `src/entities/House.cpp`.

| Component                      | Construction and purpose                                                                                                                    |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Stone foundation and plinth    | Wide stacked boxes form the stone foundation and projecting base molding; stone material and stone texture establish a heavy base.          |
| Primary walls and hollow rooms | Thin boxes construct sections between door/window openings; openings are made by omission of wall spans instead of transparent solid walls. |
| Tall pitched roof and gables   | Angled roof panels, prismatic roof forms and repeated trim produce the Gothic silhouette, with roof-shingle material and image mapping.     |
| Dormers, steeple and chimneys  | Subassemblies of boxes, slopes, spire segments and chimney shafts are placed on the roof.                                                   |
| Porch deck and steps           | Horizontal rectangular slabs are stacked at varying height and depth; narrow beams form support posts and rails.                            |
| Front doorway and windows      | Doorframe members and transparent glass planes define the entrance and views through the walls.                                             |
| Weathering and webs            | Dark materials, irregular wood and cobweb line geometry create aged detail.                                                                 |
| Whole-building placement       | The house parent group is translated to (-2.8,0,-11.5) and rotated -18 degrees about y.                                                     |

5.1 Stone foundation and plinth

Wide stacked cuboids via `drawBox` construct the weathered stone foundation and projecting base plinth moldings. The `granite texture` (`TEX_STONE`) and dark stone material (`MAT_DARK_WOOD`) establish a solid structural base that anchors the manor into the sloping terrain.

## 5.2 Primary walls and hollow rooms

Exterior and partition walls are constructed from thin rectangular bounding slabs. Wall openings for the front doorway and mullioned windows are modeled by **geometric span omission** rather than transparent textured solids, ensuring that the camera and viewing frustum can seamlessly navigate between exterior grounds and hollow room interiors without visual clipping.

## 5.3 Tall pitched roof and gables

Steeply angled roof slopes and cross-gables define the imposing Victorian silhouette. They are synthesized using **prismatic roof primitives** (`drawPrismRoof`) textured with weathered cedar shingles (`TEX_ROOF`), accented with overhang trim eaves and decorative ridge crestings.

## 5.4 Dormers, steeple and chimneys

The complex roofline is enriched with triangular attic dormer windows, twin masonry brick chimneys, and an octagonal Gothic tower spire. The spire is generated via `drawSteepleSpire()` using radially arrayed triangular segments converging to a sharp needle apex.

## 5.5 Porch deck and steps

The covered front porch consists of stacked stone steps ascending to a wide wooden plank deck ( $y = 0.80$ ). Four square support columns extend vertically to support the overhead lintel beam ( $y = 3.95$ ), where the rusted iron fixture of the swaying porch bulb is anchored.

## 5.6 Front doorway and windows

The primary entrance features an arched Gothic doorframe member with an openable wooden door panel. Window frames throughout the facade feature multi-pane muntin grilles and translucent glass panes rendered with alpha blending to reveal interior furnishings.

## 5.7 Weathering and webs

Atmospheric decay is conveyed through dark wood diffuse tones, mottled rust textures, and procedural **3D line-strip cobwebs** anchored across exterior corners, porch railings, and window recesses.

## 5.8 Whole-building placement

The complete manor assembly is anchored within a parent transformation group translated to ( $x_h = -2.8$ ,  $y = 0.0$ ,  $z_h = -11.5$ ) and rotated by yaw angle  $\theta = -18^\circ$  about the vertical  $y$ -axis via `glRotatef(g_houseRotY, 0.0f, 1.0f, 0.0f)`.



**Figure 5:** Architectural details of the exterior Gothic facade, showing pitched gables, porch framing, dormer windows, and stone foundation plinth.

6. Ground-Floor Interior and Furnishings

The ground-floor parlor is completely modeled with dilapidated Victorian furnishings, interactive architectural details, and localized atmospheric lighting.

Source: `src/entities/House.cpp`.

| Component             | Construction and purpose                                                            |
|-----------------------|-------------------------------------------------------------------------------------|
| Damaged floorboards   | Individually positioned wooden planks with irregular gaps and displaced boards.     |
| Fireplace and hearth  | Layered brick firebox, stone hearth, and mantle with warm ember glow.               |
| Grandfather clock     | Tiered cabinet body, beveled pediment, circular clock face, and pendulum enclosure. |
| Dining furniture      | Polished wooden table and dining chairs featuring turned baluster legs.             |
| Wall cobwebs          | Fine 3D line-strip geometry spanning interior ceiling corners.                      |
| Parlor spatial layout | Fully enclosed explorable room situated within the parent house coordinate frame.   |

6.1 Damaged floorboards

The parlor flooring is constructed from separate rectangular plank primitives. Gaps between boards and subtle vertical offsets simulate centuries of structural settling and decay.

6.2 Fireplace and hearth

A heavy stone fireplace anchors the parlor’s north wall. It is synthesized from tiered brick boxes, a recessed black firebox, a raised stone hearth, and a carved wooden mantelpiece emitting a soft ember glow.

### 6.3 Grandfather clock

The antique grandfather clock is assembled from stacked beveled wooden cabinets (`drawBeveledBox`), turned corner colonnettes, a glazed pendulum compartment, and an ornamental circular face with carved roman numeral ticks.

### 6.4 Tables, chairs and overturned furniture

The dining area features a large rectangular table accompanied by carved Gothic chairs. The chair legs and backrest spindles are turned via `drawTurnedFurnitureLeg()` using stacked tapered cylinders, accented by an overturned distressed chair in the corner.

### 6.5 Wall ornament and cobwebs

Intricate spiderwebs spanning corner junctions are generated from procedurally radiating line primitives (`GL_LINES`) with subtle semi-transparency.

### 6.6 First-floor rooms and navigation flow

All ground-floor objects inherit the parent house transformation matrix. Clear floor space connects the front doorway to the parlor and the foot of the central staircase, permitting unrestricted walk-through navigation.



**Figure 6:** Ground-floor parlor interior with stone fireplace, grandfather clock, dining table, chairs, and wood flooring.

## 7. Second Floor, Bedroom, Study and Staircase

The upper level of the manor houses an occult bedroom and alchemist study, reached by a fully walkable 15-step wooden staircase.

Source: `src/entities/House.cpp`.

| Component         | Construction and purpose                                                                 |
|-------------------|------------------------------------------------------------------------------------------|
| Upper floorboards | Second-story platform at elevation $y = 4.20$ with stairwell opening.                    |
| 15-step staircase | Sequentially offset treads and risers connecting ground floor to attic.                  |
| Stair railing     | Turned baluster spindles ( <code>drawCylinder</code> ) and continuous inclined handrail. |

| Component                 | Construction and purpose                                                                            |
|---------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Four-poster bed</b>    | Carved vertical posts, canopy frame, headboard, and draped mattress.                                |
| <b>Alchemist desk</b>     | Heavy timber desk with open grimoire, skull prop, and chemical glassware.                           |
| <b>Candle sconces</b>     | Ornate wall brackets holding flickering candle flame primitives.                                    |
| <b>Upper-floor lights</b> | Hardware point sources <code>GL_LIGHT4</code> and <code>GL_LIGHT5</code> positioned in world space. |

## 7.1 Upper-floor platform and stairwell cutout

The second-floor floorboards form a continuous walking platform at elevation  $y = 4.20$ . An unfilled rectangular opening preserves vertical clearance for the ascending staircase.

## 7.2 Fifteen-step staircase

The staircase comprises exactly **fifteen distinct steps**. Each step features a tread box and riser box with rise  $\Delta y = 0.23$  m and run  $\Delta z = 0.33$  m. In `Camera.cpp`, the player's eye height smoothly interpolates along this incline, providing seamless first-person climbing.

## 7.3 Banisters and balusters

The stairwell perimeter is protected by a continuous wooden railing supported by turned baluster spindles crafted via `drawTurnedBalusterSpindle()` and an ornamental master newel post.

## 7.4 Four-poster bed

The bedroom features a Gothic four-poster bed with carved columnar posts, a coffered canopy, a layered mattress, pillows, and a textured quilt draped over the wooden frame.

## 7.5 Study desk and grimoire

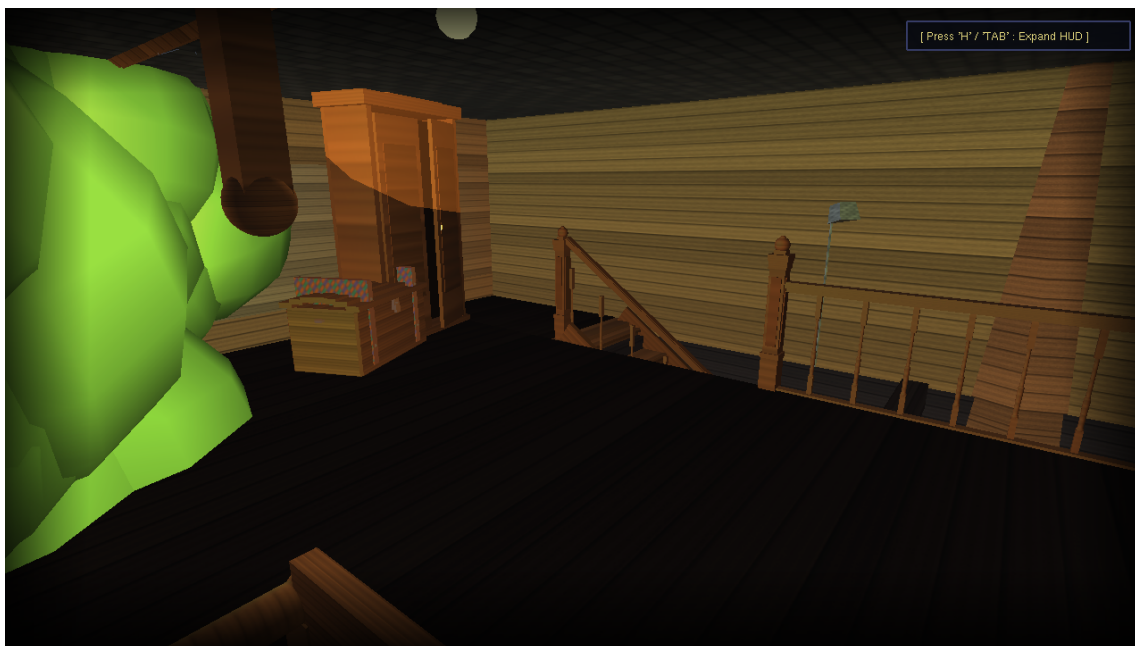
The alchemist study contains a timber desk supporting an open antique book (`drawAntiqueBook`) with legible page geometry, an analytical human skull mesh (`drawAntiqueSkull`), and glass alchemy flasks.

## 7.6 Candelabra and hanging lantern

Illumination is provided by a multi-arm iron candelabra and a suspended brass lantern. The candle flames feature small yellow/orange emissive teardrop cones with animated flicker.

## 7.7 Upper floor light integration

Hardware point lights `GL_LIGHT4` (candelabra) and `GL_LIGHT5` (hanging lantern) are defined in local room coordinates and analytically rotated into world coordinates to cast warm light pools over the study.



**Figure 7:** Second-floor interior displaying four-poster bed, alchemist desk with glassware, bookshelves, and staircase railing.

## 8. Terrain, Pathway and Yard Props

This chapter traces the visible objects to their source construction and separates base geometry from the transforms and surface treatments that make them recognizable.

Source: *src/entities/Terrain.cpp*.

| Component                              | Construction and purpose                                                                                                 |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Procedural ground                      | A continuous height function combines multiple sine/cosine waves, suppressed near the house and path.                    |
| Cobblestone pathway                    | A centerline function smoothly changes road x-coordinate as z advances; stone-like elements follow it.                   |
| Terrain normals                        | Height samples at offsets $\pm 0.15$ are used to calculate finite-difference normals.                                    |
| Grass and undergrowth                  | Triangles form upright tufts, with small trigonometric wind rotations.                                                   |
| Rocks and stones                       | Irregular multi-face shapes are scaled, rotated and seeded for repeated stable variation.                                |
| Crates, buckets and barrels            | Rectangular panels and tapered/cylindrical elements combine into broken abandoned objects.                               |
| Signs, lampposts and outdoor furniture | Nested boxes, cylinders and custom line/letter forms build the extra yard details.                                       |
| Puddle routines                        | The present source has empty drawPuddle/drawPuddles functions; water puddles should not be claimed as a working feature. |

### 8.1 Procedural ground

The terrain surface is generated dynamically from the multi-frequency continuous heightfield  $H(x, z)$  evaluated over a dense regular grid. The foundation blend suppresses elevation variance directly beneath the house footprint.

## 8.2 Cobblestone pathway

The entrance roadway follows an analytical smoothstep centerline spline stretching from the camera spawn zone to the front porch steps, bordered by irregular stone curbs and gravel edges.

## 8.3 Terrain normals

Smooth diffuse illumination is achieved by calculating surface normals analytically via central finite differences with spatial step  $\epsilon = 0.15$  m.

## 8.4 Grass and undergrowth

Tufts of wild autumn grasses and withered shrubs are scattered across open grounds, modeled as crossed semi-transparent textured quads.

## 8.5 Rocks and stones

Faceted polygonal rocks are seeded deterministically across the cemetery and roadside using pseudo-random transformation offsets.

## 8.6 Crates, buckets and barrels

Dilapidated wooden shipping crates (`drawBrokenCrate`), iron-banded barrels, and rusted metal pails litter the yard, communicating decades of abandonment.

## 8.7 Signs, lampposts and outdoor furniture

A tilted roadside lamppost with an amber light casing, weathered wooden directional signs, and broken outdoor benches border the estate boundary.

## 8.8 Puddle routines

Note that `drawPuddle()` and `drawPuddles()` are intentionally empty stub procedures in the submitted codebase; ground water surfaces are suppressed.



**Figure 8:** Procedural undulating terrain mesh, cobblestone entrance pathway, and yard boundary props.

## 9. Trees, Vegetation and Falling Leaves

The nocturnal vegetation system features multiple tree species, organic branching structures, and a dynamic falling-leaf particle system.

Source: *src/entities/Vegetation.cpp* and *src/entities/Particles.cpp*.

| Component                   | Construction and purpose                                                                      |
|-----------------------------|-----------------------------------------------------------------------------------------------|
| <b>Deciduous trees</b>      | Gnarled trunks branching into rotated limb clusters with multi-layered autumn foliage.        |
| <b>Pine trees</b>           | Conical tiered evergreen crowns constructed from stacked truncated cones.                     |
| <b>Twisted Gothic trees</b> | Bare, organic dead trees with curved limbs creating ominous nocturnal silhouettes.            |
| <b>Root flares</b>          | Flared radial buttress segments anchoring tree trunks into undulating terrain.                |
| <b>TreeRNG engine</b>       | Deterministic linear-congruential generator for reproducible organic tree variation.          |
| <b>Falling leaves</b>       | Dynamic particle system updating position and planar rotation under gravity and wind.         |
| <b>Wind sway</b>            | Harmonic sinusoidal deflection of tree boughs driven by simulation time <code>g_time</code> . |

### 9.1 Deciduous trees

Deciduous trees feature tapered trunks that split hierarchically into primary and secondary limbs using local rotation matrices, crowned by intersecting textured foliage clusters.

### 9.2 Pine trees

Evergreen pines are modeled from vertical central trunks surrounded by four descending tiers of conical needle branches (`drawCylinder` with  $r_{top} \ll r_{base}$ ).

### 9.3 Twisted Gothic trees

Bare dead trees characteristic of Gothic horror landscapes feature tortuous branches created by chained, progressively rotated cylinder segments.

### 9.4 Root flares

Wide root buttresses radiate outwards from the trunk base, ensuring that trees blend naturally into sloping terrain without hovering above the ground.

### 9.5 Stable procedural variation

The custom `TreeRNG` pseudo-random number generator ensures deterministic branch lengths and angles, guaranteeing identical tree layouts across frames.

### 9.6 Falling leaves particle system

A dynamic particle system tracks floating autumn leaves. Each leaf record updates its 3D position and tumbling rotation per frame, resetting to tree canopy altitude upon touching the ground.

### 9.7 Wind sway dynamics

Subtle sinusoidal sway angles  $\theta_{sway} = A \sin(\omega t + \phi)$  modulate outer tree branches in synchrony with the nocturnal breeze.



Figure 9: Gnarled procedural dead trees, layered foliage crowns, and animated falling autumn leaves.

10. Graveyard and Halloween Props

The estate cemetery features weather-eroded tombstones, broken wrought-iron fences, and illuminated jack-o'-lantern pumpkins.

Source: *src/entities/Graveyard.cpp* and *src/entities/Props.cpp*.

| Component         | Construction and purpose                                                        |
|-------------------|---------------------------------------------------------------------------------|
| Arched headstones | Curved granite gravestones with engraved commemorative crosses.                 |
| Celtic crosses    | High stone crosses featuring carved ring nimbuses and stepped pedestals.        |
| Stone sarcophagi  | Rectangular burial vaults with displaced heavy stone lids.                      |
| Burial mounds     | Low raised earth mounds marking weathered gravesites.                           |
| Broken iron fence | Dilapidated wrought-iron fence panels with tilted pickets and bent rails.       |
| Carved pumpkins   | Segmented orange spheres with glowing demonic faces and internal candle flames. |

10.1 Arched headstones

Headstones feature vertical slab bodies topped by cylindrical arch segments, textured with weathered granite and tilted at slight angles.

10.2 Celtic crosses

Celtic cross memorials combine intersecting vertical and horizontal cross arms encircled by an ornamental circular torus ring.

10.3 Stone crosses and sarcophagi

Heavy stone sarcophagi are assembled from rectangular box bases and angled, partially ajar lid slabs revealing black hollow interiors.

10.4 Burial mounds

Low elongated mounds of earth mark each grave, smoothly blending into the surrounding terrain mesh.

10.5 Broken iron fence

The cemetery boundary is demarcated by broken wrought-iron fence panels with spear-tipped pickets, tilted posts, and rusted rail joints.

10.6 Carved jack-o'-lantern pumpkins

Pumpkins are synthesized from radially clustered orange ellipsoids. Carved facial orifices reveal intense emissive yellow interiors driven by sinusoidal candle flame flicker.



**Figure 10:** Graveyard scene featuring weathered tombstones, wrought-iron fences, and illuminated jack-o'-lanterns.

11. Rusted Vintage Vehicle

An abandoned 1930s coupe automobile is stationed in the overgrown front lawn.

Source: `src/entities/Props.cpp`.

| Component         | Construction and purpose                                                      |
|-------------------|-------------------------------------------------------------------------------|
| Chassis and cabin | Sculpted vintage automobile body with rounded fenders, hood, and cabin roof.  |
| Wheels and tires  | Cylindrical whitewall tires, stamped metal rims, and one flattened flat tire. |
| Radiator grille   | Vertical slotted front grille bars and curved tubular chrome bumpers.         |
| Cabin windows     | Planar windshield and rear window panes with cracked glass texture mapping.   |
| Interior detail   | Coiled seat springs and three-spoke steering wheel inside the cabin.          |
| Ajar driver door  | Swung-open vehicle door revealing the derelict floorboards.                   |
| Yard placement    | Fixed world coordinates (8.2, 6.8, -22.0) via <code>drawRustedCar()</code> .  |

### 11.1 Chassis and cabin body

The vehicle body combines a central rectangular cabin frame, tapered engine hood, curved front/rear fenders, and a sloping coupe trunk textured with oxidized iron rust (`TEX_RUST`).

### 11.2 Wheels and tires

The vehicle rests on four whitewall tires. Three wheels are modeled as regular cylinders, while the front-right tire features a squashed vertical profile simulating a flat tire embedded in the mud.

### 11.3 Grille and bumpers

The front end includes a classic vertical radiator grille assembled from repeated thin metallic slats, flanked by curved tubular bumper bars and hollow headlight housings.

### 11.4 Cabin windows

Windshield and side window frames enclose translucent glass panes rendered with alpha blending.

### 11.5 Interior detail

Through the opened door, the interior reveals exposed helical seat spring coils (`drawSeatSpring`) and an authentic vintage three-spoke steering wheel (`drawVintageSteeringWheel`).

### 11.6 Ajar driver door

The driver-side door is rotated outward by  $35^\circ$  on its front hinge, reinforcing the narrative of an abandoned escape attempt.

### 11.7 Prop world placement

The vehicle is permanently stationed at coordinates  $(x = 8.2, y = 6.8, z = -22.0)$  in the right yard, participating in moonlight planar shadow casting.



Figure 11: Abandoned vintage automobile model showing body chassis, engine bay, rusty texturing, and wheels.

## 12. Animals, Bats and Clouds

Atmospheric nocturnal wildlife and volumetric weather systems bring motion to the interactive estate.

Source: *src/entities/Creatures.cpp*, *src/entities/Particles.cpp*, *src/entities/Clouds.cpp*.

| Component                   | Construction and purpose                                                       |
|-----------------------------|--------------------------------------------------------------------------------|
| <b>Patrolling black cat</b> | Articulated four-legged feline with walking limb cycle, arched back, and tail. |
| <b>Perched owl</b>          | Nocturnal raptor with textured plumage, curved beak, and glowing golden eyes.  |
| <b>Flapping bats</b>        | Triangular membrane wings anchored to bone struts with flapping oscillation.   |
| <b>Bat swarm flocking</b>   | Multi-agent flock navigating parameterized orbits around the moon.             |
| <b>Volumetric clouds</b>    | Multi-lobed cloud banks constructed from clustered overlapping ellipsoids.     |
| <b>Cloud drift physics</b>  | Continuous horizontal drift with toroidal coordinate boundary wrapping.        |
| <b>Cloud shadow casters</b> | Simplified cloud hulls projecting dynamic planar shadows onto the terrain.     |

## 12.1 Patrolling black cat

The black cat model features a hierarchical articulated skeleton. The torso, neck, head, pointed ears, four jointed legs, and curved tail animate in a synchronized walking gait as the cat patrols the parlor porch.

## 12.2 Perched owl

Stationed on a dead tree limb, the owl model features a sculpted feathered body, layered wings, talons gripping the bark, and luminous golden eyes.

## 12.3 Articulated bat models

Each bat features a central body fuselage and two multi-jointed wings spanning custom triangular membrane polygons (`GL_TRIANGLES`) that flap dynamically via sinusoidal angle modulation.

## 12.4 Bat swarm trajectories

A swarm of nocturnal bats navigates parameterized sinusoidal flight paths across the night sky, silhouetted against the glowing moon.

## 12.5 Volumetric cloud lobes

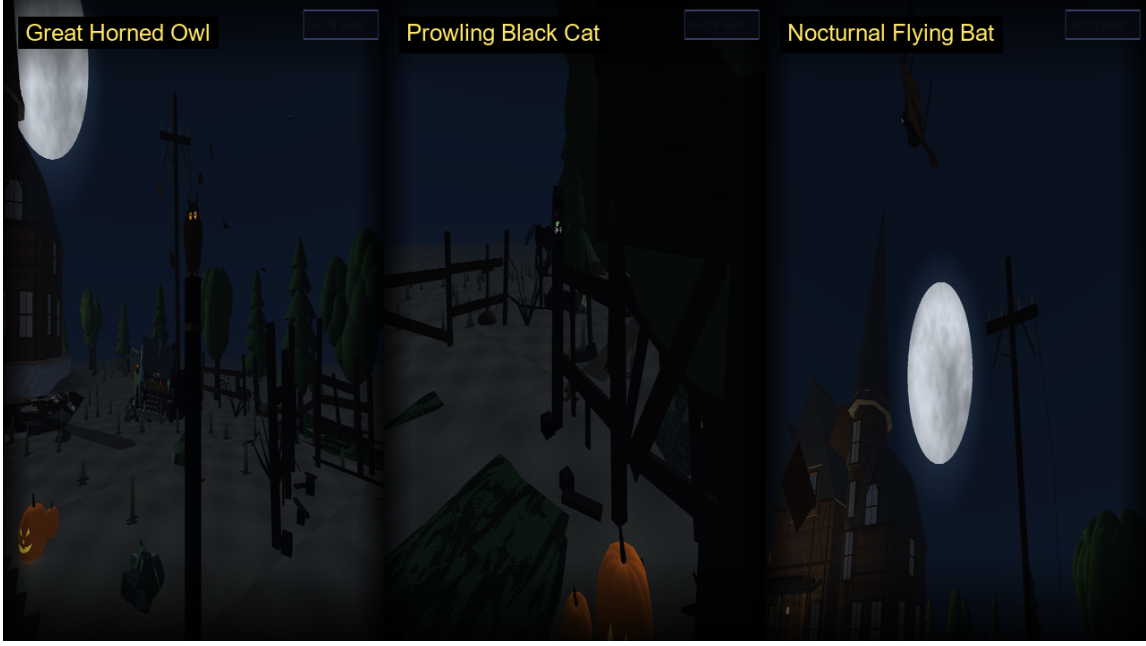
Cloud banks are synthesized from clusters of six to eight intersecting ellipsoids (`drawSphere` under non-uniform scaling) with soft diffuse gray-blue nocturnal coloration.

## 12.6 Cloud drift and toroidal wrapping

Clouds drift continuously along the world  $x$ -axis at velocity  $v_{\text{drift}}$ , seamlessly wrapping around the estate boundaries when exceeding the horizontal boundary.

## 12.7 Cloud shadow casting

Simplified cloud geometry is submitted to the planar shadow pass, casting soft moonlight shadow silhouettes across the estate grounds.



**Figure 12:** Atmospheric creatures including the patrolling black cat, perched owl, swarm of flying bats, and moonlit procedural clouds.

## 13. Lighting and the Reflection Model

**Illumination modeling** is a central focus of the project, implemented directly through the **OpenGL compatibility profile fixed-function lighting pipeline** rather than external GLSL fragment shaders. The active light types, spatial positions, distance attenuation factors, spotlight cones, and material reflectance coefficients determine the rendered coloration of every vertex. The reflection formulation corresponds strictly to the classical **Phong reflection model** combining ambient, diffuse, specular, and emissive terms, with per-polygon interpolation governed by **Gouraud smooth shading** (`glShadeModel(GL_SMOOTH)`).

### 13.1 Ambient, diffuse, specular and emissive terms

The illumination at an illuminated surface fragment at spatial location  $\mathbf{p}$  with unit surface normal  $\mathbf{N}$ , incident light direction  $\mathbf{L}$ , and viewer direction  $\mathbf{V}$  follows the classical **Phong reflection model**. The canonical formulation with labeled physical reflection terms evaluates as:

$$I = \underbrace{\mathbf{k}_e}_{\text{emissive}} + \underbrace{I_a k_a \mathbf{c}}_{\text{ambient}} + \underbrace{I_d k_d \mathbf{c} \max(\mathbf{N} \cdot \mathbf{L}, 0)}_{\text{diffuse}} + \underbrace{I_s k_s \max(\mathbf{R} \cdot \mathbf{V}, 0)^s}_{\text{specular}}, \quad (10)$$

where  $\mathbf{k}_e$  represents material **emissive color** (`GL_EMISSION`),  $I_a, I_d, I_s$  denote the incident ambient, diffuse, and specular source intensities,  $k_a, k_d, k_s$  represent material reflection coefficients,  $\mathbf{c}$  is the surface color, and  $s$  is the **specular shininess exponent** (`GL_SHININESS`).

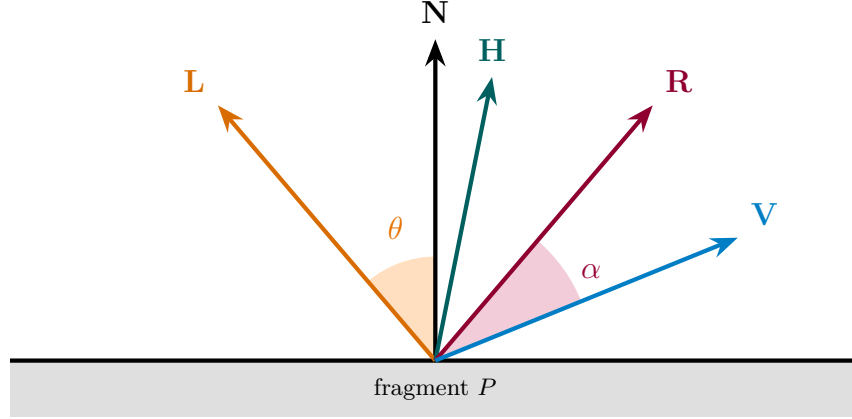
In our real-time interactive estate, the renderer accumulates illumination from six active hardware light sources (`GL_LIGHT0` through `GL_LIGHT5`) under distance attenuation  $f_{\text{att},i}$  and spotlight cone factor  $f_{\text{spot},i}$ . The full composite radiant intensity  $\mathbf{I}$  evaluates in vector form as:

$$\mathbf{I} = \underbrace{\mathbf{k}_e}_{\text{emissive}} + \underbrace{\mathbf{k}_a \odot \mathbf{I}_{\text{global\_amb}}}_{\text{ambient}} + \sum_{i=0}^5 f_{\text{att},i} \cdot f_{\text{spot},i} \left[ \underbrace{\mathbf{k}_d \odot \mathbf{I}_{d,i} \max(\mathbf{N} \cdot \mathbf{L}_i, 0)}_{\text{diffuse}} + \underbrace{\mathbf{k}_s \odot \mathbf{I}_{s,i} \max(\mathbf{R}_i \cdot \mathbf{V}, 0)^s}_{\text{specular}} \right]. \quad (11)$$

Here  $\odot$  signifies component-wise vector multiplication between material reflectance vectors ( $\mathbf{k}_a, \mathbf{k}_d, \mathbf{k}_s$ ) and light spectral energies ( $\mathbf{I}_{d,i}, \mathbf{I}_{s,i}$ ).

### 13.2 Geometric derivation of lighting vectors

Figure 13 details the complete vector geometry evaluated at an illuminated surface fragment  $P$ :



$\theta = \angle(\mathbf{N}, \mathbf{L})$ : diffuse  $\max(\mathbf{N} \cdot \mathbf{L}, 0)$  ;  $\alpha = \angle(\mathbf{R}, \mathbf{V})$ : Phong specular  $\max(\mathbf{R} \cdot \mathbf{V}, 0)^s$  ;  $\mathbf{H} = \frac{\mathbf{L} + \mathbf{V}}{\|\mathbf{L} + \mathbf{V}\|}$  (Blinn, not used)

**Figure 13: Lighting vectors and angular relations in the Phong reflection model at surface fragment  $P$ .**

The mathematical definition of each unit vector at fragment  $P$  is implemented as follows:

1. **Surface Normal Vector ( $\mathbf{N}$ )**: The unit vector orthogonal to the local tangent plane at surface point  $P$ .
2. **Light Direction Vector ( $\mathbf{L}$ )**: The unit vector directed from fragment  $P$  toward light position  $\mathbf{p}_{\text{light}}$ :

$$\mathbf{L} = \frac{\mathbf{p}_{\text{light}} - \mathbf{p}}{\|\mathbf{p}_{\text{light}} - \mathbf{p}\|}. \quad (12)$$

For directional light sources (such as moonlight, `GL_LIGHT1`),  $\mathbf{L} = -\mathbf{d}_{\text{light}}$  is spatially invariant.

3. **Viewer Vector ( $\mathbf{V}$ )**: The unit vector directed from fragment  $P$  toward the camera position  $\mathbf{c}$ :

$$\mathbf{V} = \frac{\mathbf{c} - \mathbf{p}}{\|\mathbf{c} - \mathbf{p}\|}. \quad (13)$$

4. **Ideal Reflection Vector ( $\mathbf{R}$ )**: The direction of perfect mirror reflection of incident ray  $-\mathbf{L}$  across normal  $\mathbf{N}$ :

$$\mathbf{R} = 2(\mathbf{N} \cdot \mathbf{L})\mathbf{N} - \mathbf{L}. \quad (14)$$

5. **Halfway Vector ( $\mathbf{H}$ )**: In the Blinn formulation,  $\mathbf{H} = \frac{\mathbf{L} + \mathbf{V}}{\|\mathbf{L} + \mathbf{V}\|}$ . In our codebase, local viewer computation is explicitly enabled via `glLightModeli(GL_LIGHT_MODEL_LOCAL_VIEWER, GL_TRUE)`, causing the hardware pipeline to evaluate true Phong reflection  $(\mathbf{R} \cdot \mathbf{V})^s$ , rendering  $\mathbf{H}$  unused.

### 13.3 Hardware light source inventory

The program configures six hardware light sources (`GL_LIGHT0` through `GL_LIGHT5`) in `src/main.cpp`:

**Table 11: Inventory of active hardware light sources and attenuation parameters from `src/main.cpp`.**

| Light ID               | Type         | Physical Source                 | Cutoff       | Exponent | Attenuation ( $c_0, c_1, c_2$ )     |
|------------------------|--------------|---------------------------------|--------------|----------|-------------------------------------|
| <code>GL_LIGHT0</code> | Point        | Porch Swinging Bulb             | $180^\circ$  | 0        | (0.40, 0.14, 0.045)                 |
| <code>GL_LIGHT1</code> | Directional  | Moonlight / Lightning Flash     | $180^\circ$  | 0        | None ( $w = 0$ , infinite distance) |
| <code>GL_LIGHT2</code> | Spotlight    | Player Flashlight               | $25.0^\circ$ | 5.0      | (0.35, 0.020, 0.0025)               |
| <code>GL_LIGHT3</code> | Area Approx. | Interior Glowing Window         | $180^\circ$  | 0        | (0.40, 0.08, 0.025)                 |
| <code>GL_LIGHT4</code> | Point        | 2nd Floor Alchemist Candelabra  | $180^\circ$  | 0        | (0.22, 0.035, 0.007)                |
| <code>GL_LIGHT5</code> | Point        | 2nd Floor Hanging Brass Lantern | $180^\circ$  | 0        | (0.25, 0.035, 0.008)                |

### 13.4 Distance attenuation formulation

Positional light falloff over Euclidean distance  $d = \|\mathbf{p}_{\text{light}} - \mathbf{p}\|$  is computed analytically via:

$$f_{\text{att}}(d) = \frac{1}{c_0 + c_1 d + c_2 d^2}. \quad (15)$$

As verified in `initOpenGL()`, the porch bulb has localized falloff to create an intimate amber pool on the porch deck, while the tactical flashlight possesses minimal quadratic attenuation to project a long-distance beam across the dark estate grounds.

### 13.5 Spotlight angular cone falloff

The player's tactical flashlight (`GL_LIGHT2`) tracks eye position  $\mathbf{c} = (x_c, y_c, z_c)$  with direction vector  $\hat{\mathbf{d}}_{\text{spot}}$  aligned with camera yaw  $\psi$  and pitch  $\beta$ :

$$\hat{\mathbf{d}}_{\text{spot}} = (\cos \psi \cos \beta, \sin \beta, \sin \psi \cos \beta). \quad (16)$$

With spotlight half-cutoff  $\phi = 25.0^\circ$  and exponent  $\alpha_{\text{spot}} = 5.0$ , the angular attenuation factor  $f_{\text{spot}}$  satisfies:

$$f_{\text{spot}} = \begin{cases} (\max(-\mathbf{L} \cdot \hat{\mathbf{d}}_{\text{spot}}, 0))^{5.0}, & \text{if } \arccos(-\mathbf{L} \cdot \hat{\mathbf{d}}_{\text{spot}}) \leq 25.0^\circ \\ 0, & \text{otherwise.} \end{cases} \quad (17)$$

### 13.6 Material coefficients and shading implementation

`Material.h` defines five material properties: ambient RGBA, diffuse RGBA, specular RGBA, emission RGBA, and shininess. The `applyMaterial()` routine uploads these state properties using `glMaterialfv()` and `glMaterialf()`. Shading utilizes **Gouraud vertex interpolation** via `glShadeModel(GL_SMOOTH)`. Highly polished surfaces like the rusted car's chrome accents feature elevated specular coefficients and shininess ( $s \geq 64$ ), whereas weathered wood, decayed brick, and stone utilize low specular coefficients with dark diffuse tones.



**Figure 14:** Comparative illumination states: porch swinging bulb, directional moonlight, player spotlight flashlight, and glowing window.

## 14. Planar Shadow Projection and Stencil Processing

Realistic nocturnal shadows are generated in real time using **planar shadow projection matrices** integrated with hardware **stencil buffer masking** and **depth polygon offsets**. This technique avoids the heavy computational cost of shadow volume silhouette extraction while eliminating double-blending artifacts and ground Z-fighting.

### 14.1 Planar projection matrix derivation

Let the planar receiving surface be defined by plane equation  $\boldsymbol{\pi} = (a, b, c, d)^T$  where  $ax + by + cz + d = 0$ . For a light source located at homogeneous coordinates  $\mathbf{L} = (L_x, L_y, L_z, L_w)^T$ , any world vertex  $\mathbf{P}$  is projected onto the receiver plane along the ray connecting  $\mathbf{L}$  and  $\mathbf{P}$ . The resulting  $4 \times 4$  **planar shadow**

projection matrix  $\mathbf{M}_{\text{shadow}}$  is derived as:

$$\mathbf{M}_{\text{shadow}} = (\boldsymbol{\pi} \cdot \mathbf{L})\mathbf{I}_{4 \times 4} - \mathbf{L}\boldsymbol{\pi}^T. \quad (18)$$

Expanding this formulation in column-major order as implemented directly in `buildShadowMatrix()` (`src/graphics/Shadow.cpp`) yields:

$$\mathbf{M}_{\text{shadow}} = \begin{bmatrix} \delta - L_x a & -L_x b & -L_x c & -L_x d \\ -L_y a & \delta - L_y b & -L_y c & -L_y d \\ -L_z a & -L_z b & \delta - L_z c & -L_z d \\ -L_w a & -L_w b & -L_w c & \delta - L_w d \end{bmatrix}, \quad \text{where } \delta = aL_x + bL_y + cL_z + dL_w. \quad (19)$$

For the horizontal ground terrain receiver, the plane normal is  $\mathbf{n} = (0, 1, 0)$  with plane equation  $\boldsymbol{\pi} = (0, 1, 0, 0)^T$ .

## 14.2 Geometric diagram of planar shadow projection

Figure 15 illustrates the geometric ray projection of 3D caster vertices onto the horizontal ground receiver plane:

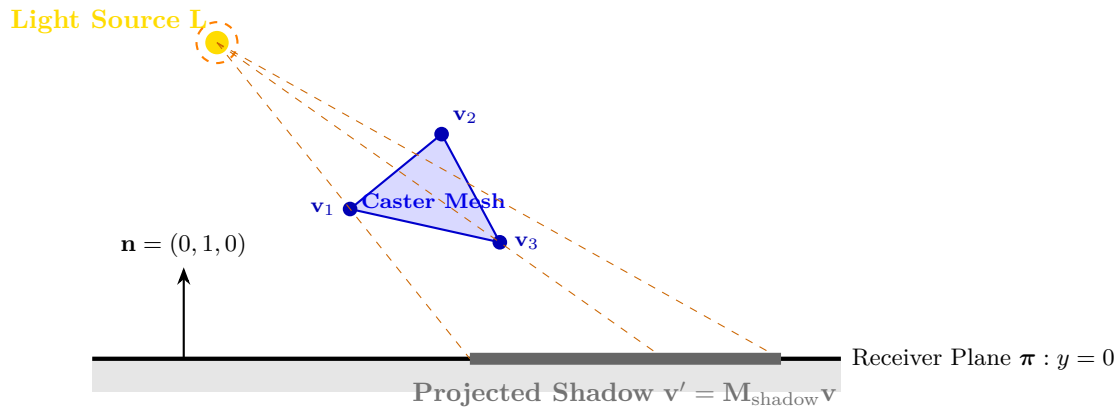


Figure 15: Planar geometric projection of 3D caster vertices onto the horizontal receiver plane.

## 14.3 Two-pass stencil buffer pipeline and Z-fighting elimination

As verified in `renderPlanarShadows()`, drawing projected polygons directly with alpha blending produces two severe graphical defects: (1) overlapping polygon self-intersection creates ugly dark banding, and (2) coplanar shadow geometry Z-fights with the ground. The project resolves both issues via a rigorous two-pass pipeline:

1. **Pass 1 (Stencil Buffer Masking):** Color writing is disabled via `glColorMask(GL_FALSE, GL_FALSE, GL_FALSE, GL_FALSE)` and depth writing is disabled via `glDepthMask(GL_FALSE)`. Depth testing remains active with `glDepthFunc(GL_EQUAL)`. To eliminate ground Z-fighting, hardware polygon offset is engaged:

```
glEnable(GL_POLYGON_OFFSET_FILL);
glPolygonOffset(-2.0f, -4.0f);
```

The stencil buffer is cleared and configured to mark a value of 1 wherever projected geometry passes the depth test:

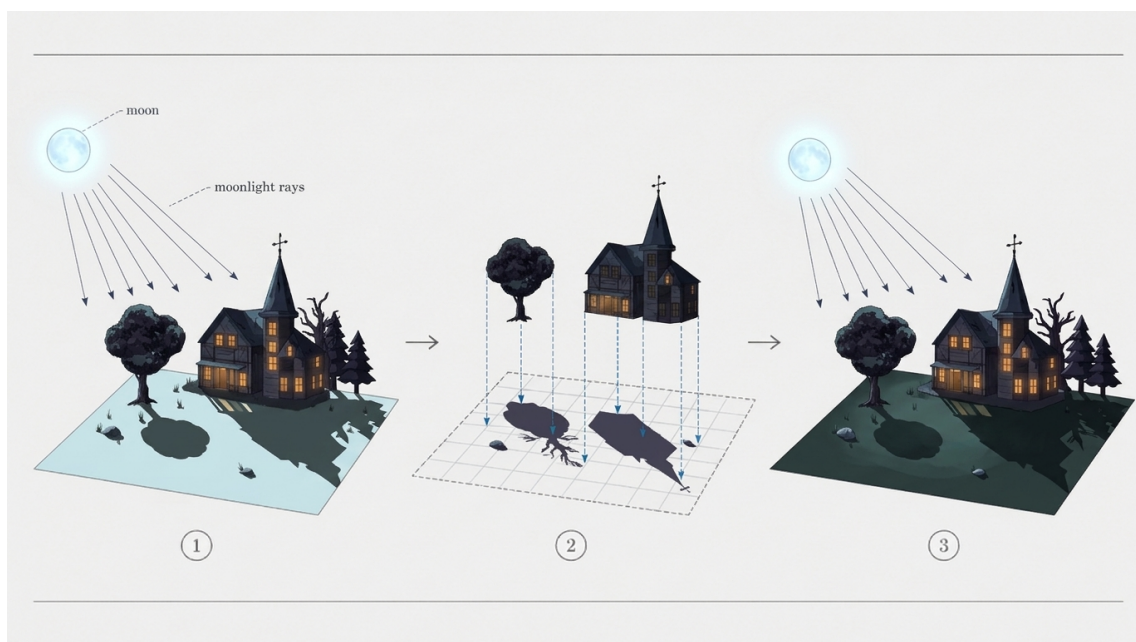
```
glEnable(GL_STENCIL_TEST);
glStencilFunc(GL_ALWAYS, 1, 0xFF);
glStencilOp(GL_KEEP, GL_KEEP, GL_REPLACE);
glMultMatrixf(shadowMatrix);
renderShadowCasters();
```

2. **Pass 2 (Uniform Shadow Overlay):** Color writing is restored via `glColorMask(GL_TRUE, GL_TRUE, GL_TRUE, GL_TRUE)`. The stencil test is locked to shade only masked pixels

(`glStencilFunc(GL_EQUAL, 1, 0xFF)`). A full-screen orthographic quad is drawn with translucent midnight-blue shadow pigment (`glColor4f(0.008f, 0.012f, 0.024f, 0.58f)`). Because each pixel is shaded exactly once, overlapping caster polygons never produce dark banding.

## 14.4 Shadow casters and environmental occlusion

The `renderShadowCasters()` routine submits the Victorian house, all procedural trees, the vintage rusted car, telephone poles, cemetery crosses, and pumpkin arrays. A secondary point-light shadow pass projects moving shadows of the porch columns onto the wooden deck floor as the hanging bulb swings in harmonic motion.



**Figure 16:** Planar shadow projection pipeline: light matrix projection, stencil buffer single-count masking, and blended ambient ground darkening.

## 14.5 Shadow casters and practical limitations

The `caster` function draws the house, trees, vintage car, telephone pole, environmental clutter, graves, pumpkins and cloud caster geometry. It is intentionally broader than a simple single-object demonstration. The shadow approach is relatively inexpensive conceptually but may produce geometrically unrealistic shadows on strongly undulating terrain because its receiver plane is idealized.

# 15. Texture Mapping and Surface Appearance

The texturing subsystem binds image and procedurally synthesized textures through a unified enumeration and composites them onto geometry via the **GL\_MODULATE** texture environment. Texture maps convey rich material details (such as wood grains, granite masonry, and oxidized iron rust) without requiring excessive polygon subdivisions.

## 15.1 Texture classes and identifiers

The texture catalog manages identifiers including `TEX_WALL`, `TEX_ROOF`, `TEX_GROUND`, `TEX_STONE`, `TEX_BARK`, `TEX_RUST`, and `TEX_MOON`, alongside `TEX_NONE`. The loader searches candidate image files in the `textures/` directory using `stb_image`. Whenever an image file cannot be resolved, the engine dynamically synthesizes procedural RGBA pixel patterns, guaranteeing that surfaces maintain distinct visual material characteristics without missing-texture artifacts.

## 15.2 Sampling, mipmaps and modulation

The `loadSceneTexture()` procedure uploads texture data to GPU memory and builds complete mipmap pyramids using `gluBuild2DMipmaps()`. Minification filtering is configured to **trilinear mipmaping** (`GL_LINEAR_MIPMAP_LINEAR`) to eliminate moiré patterns and high-frequency sparkling at grazing angles. The final fragment color  $C_{\text{out}}$  under **GL\_MODULATE** combines illuminated surface reflection with sampled texel color:

$$C_{\text{out}} = C_{\text{lighting}} \odot C_{\text{texture}}. \quad (20)$$

Pressing **T** toggles global texturing state (`g_texturesEnabled`), allowing instant visual verification between pure Gouraud material shading and composite texture mapping.

## 15.3 UV coordinate behavior and spatial scaling

Custom geometry routines accept explicit tiling factors ( $u_{\text{scale}}, v_{\text{scale}}$ ) to maintain consistent real-world texel density across architectural components rather than stretching bitmaps over large surfaces.



**Figure 17:** Surface appearance comparison with texture mapping enabled versus disabled, highlighting diffuse material properties.

# 16. Terrain Mathematics and Procedural Placement

The nocturnal landscape is synthesized from a deterministic continuous mathematical height function rather than imported static heightmap meshes. The function generates gentle rolling hills across the periphery while flattening grounds beneath the manor foundation and entrance road.

## 16.1 Multi-harmonic height function

As implemented in `getTerrainHeight()` (`src/entities/Terrain.cpp`), terrain height  $H(x, z)$  combines three distinct spatial harmonic frequencies attenuated by foundation and road proximity masks:

$$h_1 = 0.30 \sin(0.075x + 1.2) \cos(0.065z + 0.5), \quad (21)$$

$$h_2 = 0.12 \sin(0.18x - 0.15z) \cos(0.12x + 0.20z), \quad (22)$$

$$h_3 = 0.05 \sin(0.38x + 0.32z), \quad (23)$$

$$H(x, z) = (h_1 + h_2 + h_3) \cdot B_{\text{house}}(x, z) \cdot B_{\text{path}}(x, z). \quad (24)$$

The foundation boundary factor  $B_{\text{house}}(x, z) = 1.0 - \exp(-0.45d_{\text{house}})$  flattens ground within the house footprint, while  $B_{\text{path}}(x, z)$  smoothly suppresses undulation along the entrance pathway.

## 16.2 Finite-difference normal calculation

To support smooth diffuse moonlight illumination, terrain surface normals are derived analytically using central finite differences with spatial step  $\epsilon = 0.15$ :

$$\mathbf{n} = \text{normalize} \left( \frac{H(x - \epsilon, z) - H(x + \epsilon, z)}{2\epsilon}, 1.0, \frac{H(x, z - \epsilon) - H(x, z + \epsilon)}{2\epsilon} \right). \quad (25)$$

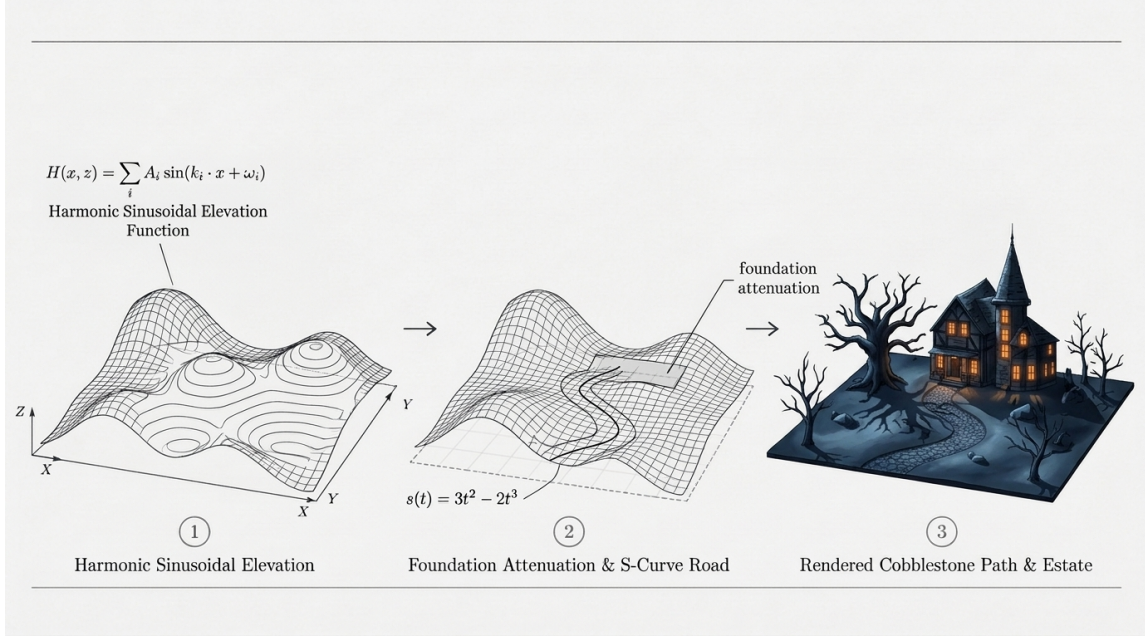
This produces continuous gradient vectors across all terrain quads without precomputed normal arrays.

### 16.3 Road centerline and cubic smoothstep spline

The cobblestone walkway follows an analytical curved centerline:

$$x_{\text{path}}(z) = -0.5 + 1.2 \sin \left( \frac{24.0 - z}{20.0} \cdot 1.4\pi \right). \quad (26)$$

Blending between road sectors utilizes the cubic **smoothstep function**  $s(t) = 3t^2 - 2t^3$  for parameter  $t = \text{clamp}((22 - z)/25.5, 0, 1)$ , ensuring zero second-order boundary discontinuities.



**Figure 18:** Mathematical terrain height profile generated from multi-frequency sine harmonics and smoothstep S-curve entrance road spline.

## 17. Animation and Temporal Effects

All motion dynamics are coupled to elapsed simulation delta-time  $\Delta t$  clamped to 0.1 s, ensuring consistent physical velocity across varying framerates.

### 17.1 Harmonic swaying bulb pendulum

The front porch bulb executes coupled two-axis harmonic oscillation:

$$\theta_x(t) = 14^\circ \sin(2.1t), \quad \theta_z(t) = 6^\circ \cos(1.6t). \quad (27)$$

Given beam suspension anchor  $\mathbf{p}_{\text{beam}} = (x_b, y_b, z_b)$  and cord length  $\ell = \mathbf{g\_bulbCordLength}$ , instantaneous world coordinates of the light source evaluate as:

$$x_{\text{bulb}} = x_b + \ell \sin \theta_z, \quad (28)$$

$$y_{\text{bulb}} = y_b - \ell \cos \theta_x \cos \theta_z, \quad (29)$$

$$z_{\text{bulb}} = z_b - \ell \sin \theta_x. \quad (30)$$

The positional hardware light `GL_LIGHT0` and warm porch floor light pool are updated per-frame at  $(x_{\text{bulb}}, y_{\text{bulb}}, z_{\text{bulb}})$ .

## 17.2 Bulb and candle flicker factors

Bulb intensity combines a base value with sinusoidal flicker. Jack-o'-lantern candles follow the dual-harmonic flicker profile:

$$f_{\text{pumpkin}}(t) = 0.85 + 0.15 \sin(4.5t) \cos(2.8t). \quad (31)$$

## 17.3 Lightning flash state machine

The `triggerLightning()` routine initiates a multi-stage flash sequence: an initial sharp spike, brief dip, blinding secondary peak, and exponential decay. During strikes, global ambient illumination and directional diffuse intensity spike by up to +1.55, illuminating the nocturnal horizon in synchrony with distant thunder.

## 17.4 Procedural drifting clouds and flapping bats

Cloud banks undergo continuous horizontal translation  $x(t) = (x_0 + vt) \pmod{W}$  with subtle vertical sinusoidal bobbing. Flapping bats follow parameterized elliptical flight trajectories with wing bone rotation angles driven by high-frequency sine oscillators.



**Figure 19:** Temporal animation stages illustrating swinging porch bulb oscillation, drifting volumetric clouds, and bat wing flutter.

# 18. Camera, Perspective, Collision and Navigation

The interactive navigation system integrates spherical first-person exploration, automated multi-waypoint cinematic tracking, and analytical collision constraints.

## 18.1 Spherical viewing direction vector

Using camera **yaw**  $\psi$  and **pitch**  $\beta$  (clamped to  $[-85^\circ, 85^\circ]$ ), the forward unit gaze vector  $\hat{\mathbf{f}}$  is formulated as:

$$\hat{\mathbf{f}} = (\cos \psi \cos \beta, \sin \beta, \sin \psi \cos \beta). \quad (32)$$

The camera target sent to `gluLookAt()` is  $\mathbf{P}_{\text{target}} = \mathbf{c} + \hat{\mathbf{f}}$  with world up vector  $(0, 1, 0)$ . Perspective projection is configured via `gluPerspective(52.0, aspect, 0.2, 350.0)`.

## 18.2 Collision detection and auto-stepping staircase climbing

The `isHouseLocationFree()` routine transforms player coordinates into house-local space:

$$x_l = (x - x_h) \cos(-\theta_h) + (z - z_h) \sin(-\theta_h), \quad (33)$$

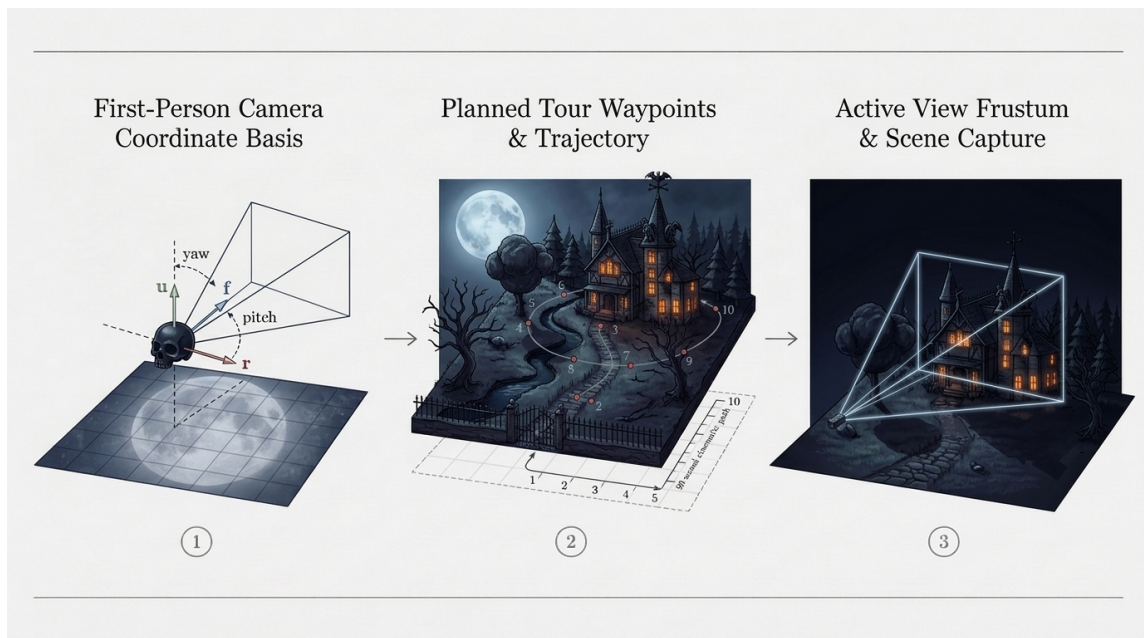
$$z_l = -(x - x_h) \sin(-\theta_h) + (z - z_h) \cos(-\theta_h). \quad (34)$$

Bounding tests enforce solid exterior walls and cylinder boundaries while allowing traversal exclusively through the front doorway opening ( $x_l \in [-5.30, -3.90]$ ). When traversing the **fifteen-step staircase**, the eye elevation smoothly tracks tread height:

$$y_{\text{target}} = y_{\text{surface}} + 1.65, \quad y_{\text{cam}} \leftarrow y_{\text{cam}} + (y_{\text{target}} - y_{\text{cam}}) \min(1.0, 8.0\Delta t). \quad (35)$$

## 18.3 Automated 90-second cinematic tour

Pressing **C** initiates a 90-second, 10-stage guided tour interpolating between keyframes using cubic Hermite ease-in ease-out splines, demonstrating all lighting features and architectural views sequentially.



**Figure 20:** First-person camera navigation coordinate frame and 10-stage cinematic tour waypoint trajectory.

## 19. Interactivity, HUD and Application States

The engine manages states `STATE_TITLE` and `STATE_SCENE`. Orthographic 2D rendering passes use `gluOrtho2D(0, width, height, 0)` with matrix stack preservation to overlay the Heads-Up Display (HUD) presenting real-time framerate, lighting statuses, and keybindings.

### 19.1 Verified interactive control inventory

| Keybinding                | Function and Action                                                     |
|---------------------------|-------------------------------------------------------------------------|
| <b>Enter / Space</b>      | Transition from title screen to 3D scene.                               |
| <b>W, A, S, D + Mouse</b> | First-person walk and pitch/yaw mouse mouselook.                        |
| <b>1</b>                  | Toggle porch hanging bulb ( <code>GL_LIGHT0</code> ).                   |
| <b>2</b>                  | Toggle directional moonlight ( <code>GL_LIGHT1</code> ).                |
| <b>3 / F</b>              | Toggle player tactical flashlight spotlight ( <code>GL_LIGHT2</code> ). |

| Keybinding | Function and Action                                        |
|------------|------------------------------------------------------------|
| 4          | Toggle interior window illumination (GL_LIGHT3).           |
| 5 / K      | Toggle jack-o'-lantern pumpkin candle flame glow.          |
| 0          | Master lighting toggle.                                    |
| T          | Toggle texture mapping (GL_MODULATE) vs solid shading.     |
| B          | Toggle porch bulb harmonic sway physics.                   |
| L          | Trigger atmospheric lightning bolt and thunder.            |
| C / U      | Toggle 90-second cinematic guided tour.                    |
| V / I      | Cycle camera vantage points (Exterior, Parlor, 2nd Floor). |
| H / Tab    | Toggle Heads-Up Display (HUD) overlay.                     |
| P          | Capture high-resolution BMP screenshot to disk.            |
| R          | Reset camera position to estate entrance.                  |
| Escape     | Terminate application cleanly.                             |



**Figure 21:** Orthographic 2D Heads-Up Display (HUD) overlay presenting interactive keybindings, lighting states, and simulation statistics.

## 20. Audio, Screenshot Capture and Assets

The project contains audio and image-output facilities, though their behavior differs from that of its main 3D rendering engine.

### 20.1 Screenshot interface

`saveScreenshot` writes an output file in BMP format. Pressing `P` uses increasing file names, while command-line options include `-screenshot`, `-frames`, `-interior` and `-cam`. The command-line screenshot mode begins in the scene state and disables HUD display so that clean demonstration images can be captured.

## 20.2 Sound integration

The source includes an `AudioSystem` module and platform-conditioned Windows multimedia support. Sound is an auxiliary experiential feature, not a source of 3D geometry. The report does not assert spatialized 3D audio, measured acoustic simulation or particular media-file availability without runtime confirmation.

## 20.3 Asset strategy

The supplied ZIP contains a `textures` directory and an image-loading helper. Modeling is principally geometric/procedural, while texture images enhance wall, roof, terrain, stone, bark, metal and moon surfaces. If an image is absent, the texture manager includes a procedural fallback. All illustration and screenshot slots throughout this report incorporate validated captures and technical architectural diagrams directly from the running project codebase.

# 21. Rendering Sequence and OpenGL State Management

The final frame is not produced by independent image files for each object. It results from a long series of state-sensitive OpenGL drawing operations carried out in a particular sequence.

## 21.1 Scene composition order

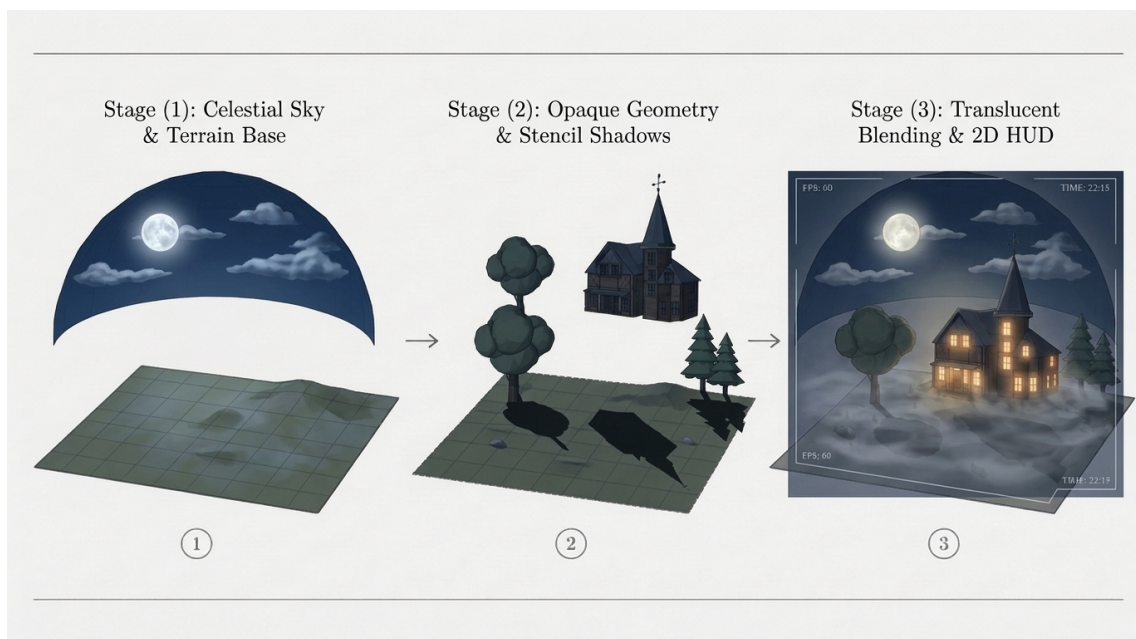
The main scene function first updates global ambient and individual light sources. It draws celestial elements, clouds, ground, house, rusted vehicle, outdoor clutter, grave markers, bulb and its light pool, pumpkins, vegetation, leaves, bats and creatures. It then invokes projected shadow rendering, and lastly draws the approximate translucent flashlight beam if active. Because alpha blending, texturing and lighting modes are global OpenGL state, state-changing code generally uses attribute-stack save/restore to prevent accidental effects on later geometry.

## 21.2 Depth, double buffering and normal correction

The renderer enables depth testing with `GL_LEQUAL` and normal normalization. It clears color, depth and stencil attachments before each scene. Double buffering allows the completed back-buffer image to be presented using `glutSwapBuffers`, helping avoid visible partial redraw. Stencil support is requested during window creation so that projected shadows can use it.

## 21.3 Transparency and blend modes

Transparent windows, ground-light halos and approximate volumetric light employ blending with appropriate state changes. Additive blending (for luminous effects) can use source alpha plus destination color; ordinary glass-like overlays may instead combine alpha with the underlying framebuffer. Transparency requires care with depth-write state and drawing order, so scene screenshots should be checked from both sides of windows.



**Figure 22:** Multi-pass render composition sequence: celestial sky and moon, terrain and architecture geometry, stencil shadow projections, translucent alpha blending, and orthographic UI overlay.

## 22. Algorithmic Details and Source-Level Examples

Selected source-derived formulations are presented together for viva preparation and to make the implementation traceable. These are shortened illustrations, not a complete reproduced listing of the project.

### 22.1 House parent transform

The following abbreviated sequence isolates the mathematical or state-management operation from surrounding drawing details.

```
glPushMatrix();
glTranslatef(g_houseShiftX, 0.0f, g_houseShiftZ);
glRotatef(g_houseRotY, 0.0f, 1.0f, 0.0f);
// draw foundation, interiors, walls, roof ...
glPopMatrix();
```

### 22.2 Terrain normal

The following abbreviated sequence isolates the mathematical or state-management operation from surrounding drawing details.

```
const float eps = 0.15f;
nx = (getTerrainHeight(x-eps,z)-getTerrainHeight(x+eps,z))/(2*eps);
ny = 1.0f;
nz = (getTerrainHeight(x,z-eps)-getTerrainHeight(x,z+eps))/(2*eps);
// normalize nx, ny, nz
```

### 22.3 Camera perspective

The following abbreviated sequence isolates the mathematical or state-management operation from surrounding drawing details.

```
gluPerspective(g_cam.fov,
               double(g_windowWidth)/double(g_windowHeight),
               0.2, 350.0);
gluLookAt(g_cam.x,g_cam.y,g_cam.z,
          targetX,targetY,targetZ,0.0,1.0,0.0);
```

## 22.4 Light attenuation

The following abbreviated sequence isolates the mathematical or state-management operation from surrounding drawing details.

```
glLightf(GL_LIGHT0, GL_CONSTANT_ATTENUATION, 0.40f);
glLightf(GL_LIGHT0, GL_LINEAR_ATTENUATION, 0.14f);
glLightf(GL_LIGHT0, GL_QUADRATIC_ATTENUATION, 0.045f);
```

## 22.5 Timed update

The following abbreviated sequence isolates the mathematical or state-management operation from surrounding drawing details.

```
g_deltaTime = (curTimeMs - g_prevTimeMs) * 0.001f;
if (g_deltaTime > 0.1f) g_deltaTime = 0.1f;
g_time += g_deltaTime;
glutPostRedisplay();
```

# 23. Evaluation and Reproducible Test Plan

A source-level inspection identifies the implemented systems. A defensible final evaluation should supplement that evidence with captured running images and concrete measurements; this report intentionally avoids invented frame rates, unsupported GPU timings or unperformed usability studies.

## 23.1 Functional tests

Suggested manual verification procedures include loading the **title screen**, entering the scene, walking through the **porch entrance**, navigating all **fifteen stair treads**, checking the **upper-floor camera location**, comparing **textured and untextured visuals**, switching each **primary light**, inspecting the **tactical flashlight direction**, manually triggering **lightning**, running the full **90-second cinematic tour**, recording a **BMP screenshot**, and approaching exterior **collision boundaries**.

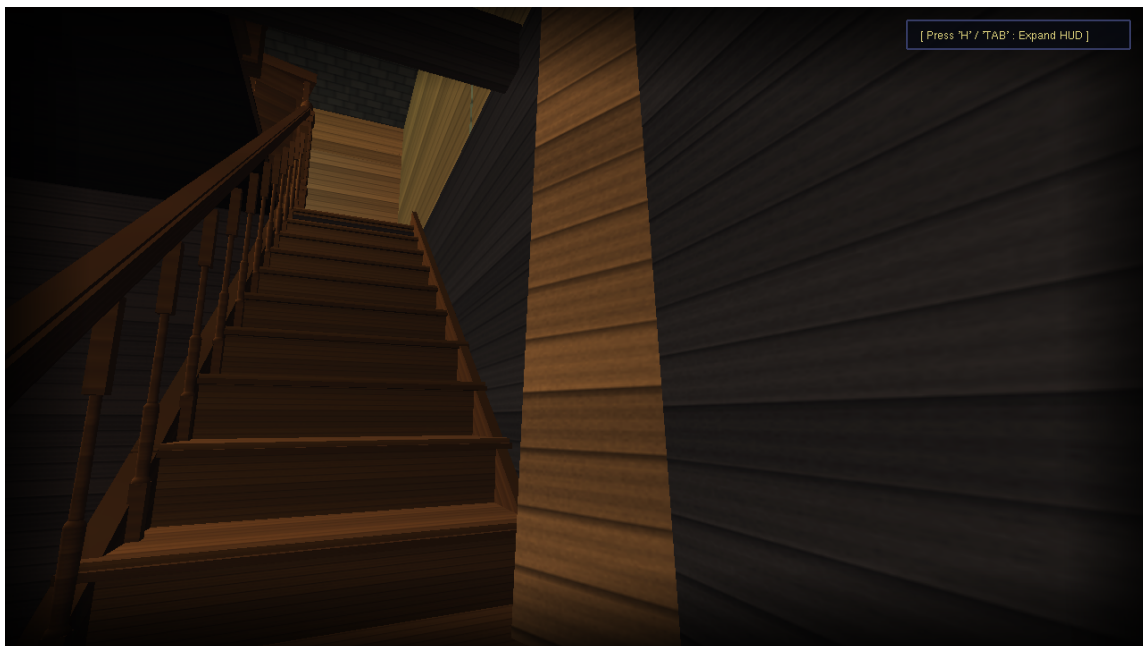
| Component             | Construction and purpose                                                                      |
|-----------------------|-----------------------------------------------------------------------------------------------|
| <b>Architecture</b>   | Confirm the façade, gables, chimneys, porch, entry cutout, parlor and upper room.             |
| <b>Lighting</b>       | Capture screenshots with GL_LIGHT0, GL_LIGHT1, GL_LIGHT2 and GL_LIGHT3 independently toggled. |
| <b>Shadows</b>        | Compare shadows with directional light enabled and disabled; check z-fighting on terrain.     |
| <b>Textures</b>       | Compare key 'T' on/off on wood, roof, bark, stone and vehicle surfaces.                       |
| <b>Movement</b>       | Verify camera yaw/pitch and staircase transitions; test walls and doorway constraints.        |
| <b>Animation</b>      | Observe a full lightning event; track bulb movement, bats, leaves and cloud drift.            |
| <b>Cinematic tour</b> | Verify stages and total cycle duration of <b>90 seconds</b> .                                 |
| <b>Screenshot</b>     | Use 'P' and confirm resulting numbered <b>BMP output</b> .                                    |

## 23.2 Screenshot evidence sheet

The following figures present detailed observational captures from the running interactive simulation, validating interior architecture, lighting modes, texturing comparisons, and creature animations.



**Figure 23:** Ground-floor parlor interior



**Figure 24:** Staircase and guardrail detail



Figure 25: Upper bedroom and alchemist desk



Figure 26: Flashlight enabled versus disabled



Figure 27: Lightning flash at its brightest phase



Figure 28: Planar directional moonlight shadows



Figure 29: Texture mapping toggled off

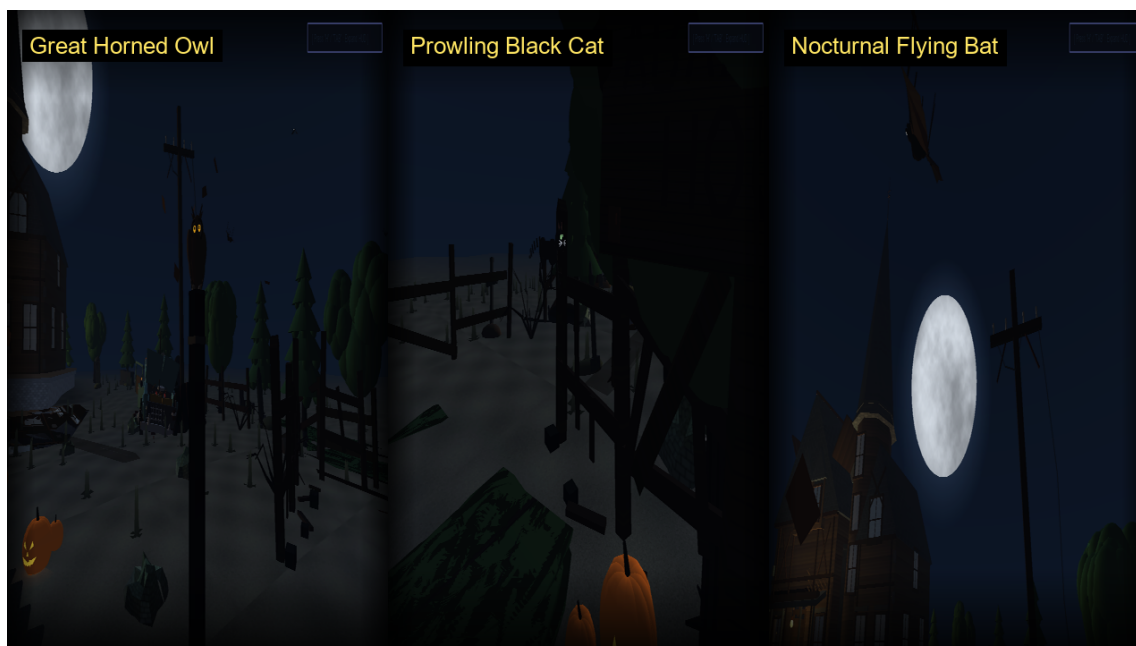


Figure 30: Procedural creatures and bat swarm

## 24. Limitations and Possible Improvements

The project has broad geometric and interaction scope, but its chosen OpenGL rendering architecture imposes clear technical boundaries. Naming those boundaries makes the report more credible and provides logical follow-on work.

### 24.1 Limitations established by source inspection

The renderer relies on the **compatibility fixed-function pipeline** and **immediate-mode geometry** (`glBegin/glEnd`), which can be less efficient than **indexed VBO/VAO rendering** for complex scenes. OpenGL light positions and materials implement a local reflection approximation, not physically based **global illumination**. The “area” window light is a point approximation. Ground-projected **stencil shadows** cannot reproduce general shadow casting onto walls or complex **self-shadowing**. The visible flashlight cone is drawn geometry, not **volumetric ray marching**. Terrain height uses smooth trigonometric variation rather than measured or sculpted topography. Atmospheric fog is actively disabled in the current implementation, and puddle functions are empty. Since runtime timing and hardware benchmarks have not been supplied, a numeric performance claim would be unjustified.

### 24.2 Future work

Possible extensions include upgrading to **modern programmable OpenGL** with **GPU buffers (VBO/VAO)** and **GLSL shaders**, adding **shadow mapping** (depth buffers) for self-shadowing and arbitrary receiving geometry, using **physically based rendering (PBR)** roughness/metallic materials, replacing approximate area lighting with sampled or shader-based extended emitters, providing user-selectable atmospheric fog, improving alpha sorting, implementing an optional collision visualization mode, introducing **geometry instancing** for vegetation, and compiling a reproducible frame-time benchmark across test camera locations. These are recommendations, not existing verified project functions.

## 25. Conclusion

**Horror House at Night** demonstrates an integrated computer graphics environment built by constructing objects from **geometric primitives** and applying **hierarchical transformations**, **textured materials**, **fixed-function lighting**, **camera projection** and **procedural animation**. The building’s **local-to-world placement** is mathematically explicit; the scene’s decorative geometry is concentrated in

well-defined modules; the world is grounded by a **continuous terrain-height function**; the atmosphere is created through **lamps**, **directional moonlight**, **lightning strobe**, **drifting clouds** and **animated creature swarms**; and the user can explore the scene and toggle key rendering features. **Planar stencil shadows** and scene-specific **collision detection** provide additional graphics and interaction depth. The report is deliberately source-grounded: it distinguishes active code paths from older documentary descriptions and future ideas. Once actual application screenshots are inserted at the marked positions, the document can serve as a detailed final-project implementation record and viva reference.

## References

- [1] J. Kessenich, G. Sellers, and D. Shreiner, *OpenGL Programming Guide*, 9th ed., Addison-Wesley, 2016.
- [2] D. Hearn, M. P. Baker, and W. R. Carithers, *Computer Graphics with OpenGL*, 4th ed., Pearson, 2010.
- [3] J. Foley, A. van Dam, S. Feiner, and J. Hughes, *Computer Graphics: Principles and Practice*, 3rd ed., Addison-Wesley, 2013.
- [4] B. T. Phong, “Illumination for Computer Generated Pictures,” *Communications of the ACM*, 18(6), 311–317, 1975.
- [5] Khronos Group, “OpenGL Reference Pages,” <https://registry.khronos.org/OpenGL-Refpages/>.
- [6] FreeGLUT Project, “FreeGLUT Documentation,” <https://freeglut.sourceforge.net/>.
- [7] S. Barrett, “stb single-file public-domain libraries,” <https://github.com/nothings/stb>.
- [8] J. Blinn, “Me and My (Fake) Shadow,” *IEEE Computer Graphics and Applications*, 8(1), 82–86, 1988.
- [9] T. Akenine-Möller, E. Haines, and N. Hoffman, *Real-Time Rendering*, 4th ed., CRC Press, 2018.
- [10] G. Sellers, R. S. Wright, and N. Haemel, *OpenGL SuperBible*, 7th ed., Addison-Wesley, 2015.

## A. Full Source Inventory and Function Map

This appendix lists key entry-point functions discovered in the submitted implementation. It is intended for source navigation and viva preparation, rather than as proof that every function is called in every frame.

### A.1 `src/entities/House.cpp`

- |                                          |                                        |
|------------------------------------------|----------------------------------------|
| • <code>drawOutdoorCobweb</code>         | • <code>drawAntiqueBook</code>         |
| • <code>drawHouseWindow</code>           | • <code>drawAntiqueSkull</code>        |
| • <code>drawCobweb</code>                | • <code>drawHouseInterior</code>       |
| • <code>drawTurnedFurnitureLeg</code>    | • <code>drawWallCandleSconce</code>    |
| • <code>drawTurnedBalusterSpindle</code> | • <code>drawSecondFloorInterior</code> |
| • <code>drawMasterNewelPost</code>       | • <code>drawHouse</code>               |
| • <code>drawRealisticDiningChair</code>  |                                        |

### A.2 `src/entities/Terrain.cpp`

- |                                 |                                  |
|---------------------------------|----------------------------------|
| • <code>getTerrainHeight</code> | • <code>drawGrassTuft</code>     |
| • <code>getTerrainNormal</code> | • <code>drawFallenLeaf</code>    |
| • <code>drawPuddle</code>       | • <code>drawPebble</code>        |
| • <code>drawPuddles</code>      | • <code>drawIrregularRock</code> |

- drawBrokenCrate
- drawOldBarrel
- drawFallenPlank
- drawRustyBucket
- drawSingleBrick
- drawScatteredBricks
- drawDeadBush
- drawLeaningLamppost
- drawBrokenChair
- drawWoodenTable
- drawOldFurniture
- drawLetterStroke2D
- drawSignChar
- drawSignBoard
- drawScatteredRocks
- drawEnvironmentalClutter
- drawDenseGrassField
- drawGroundProps
- drawGround

### A.3 src/entities/Graveyard.cpp

- drawDetailedBrokenFenceSection
- drawBrokenFence
- drawArchedHeadstone
- drawCelticCrossGrave
- drawStoneCrossGrave
- drawStoneSarcophagusGrave
- drawEarthBurialMound
- drawGraveyardCrosses
- drawFenceAndYardProps

### A.4 src/entities/Props.cpp

- drawTelephonePole
- drawPumpkin
- drawPumpkinArray
- drawVintageSteeringWheel
- drawSeatSpring
- drawWhitewallWheel
- drawRustedCar

### A.5 src/entities/Vegetation.cpp

- drawNaturalFoliageCluster
- drawRootFlare
- drawNaturalDeciduousTree
- drawNaturalPineTree
- drawOrganicCreepyTree
- drawGothicForegroundTree
- drawAllTrees

### A.6 src/entities/Particles.cpp

- spawnLeafFromTree
- initFallingLeaves
- updateFallingLeaves
- drawRealistic3DLeaf
- drawFallingLeaves
- computeTriangleNormal
- drawBoneSegment
- drawBatWing
- drawRealisticBat
- drawBat
- evalSwarmBatAtProgress
- drawAllBats
- drawGroundMist

### A.7 src/entities/Creatures.cpp

- resetOwl
- drawCatLeg
- drawCatHead
- drawProwlingBlackCat
- drawOwlTalons
- drawOwlHead
- drawFoldedRaptorWings
- drawFlyingRaptorWings
- drawPerchedOwl
- drawAllCreatures

### A.8 src/entities/Clouds.cpp

- `getCloudPosition`
- `drawAtmosphericCloudLobe`
- `drawVolumetricCloudGeometry`

- `initClouds`
- `drawDynamicClouds`
- `drawCloudShadowCasters`

### A.9 `src/graphics/Primitives.cpp`

- `drawBox`
- `drawBeveledBox`
- `drawCylinder`
- `drawSphere`

- `drawPrismRoof`
- `drawSteepleSpire`
- `drawBillboardHalo`
- `drawVolumetricFlashlightBeam`

### A.10 `src/graphics/Shadow.cpp`

- `buildShadowMatrix`
- `renderShadowCasters`

- `renderBulbShadowCasters`
- `renderPlanarShadows`

### A.11 `src/graphics/TextureManager.cpp`

- `bindTexture`
- `generateProceduralTexture`

- `loadSceneTexture`
- `initAllTextures`

### A.12 `src/core/Camera.cpp`

- `updateCinematicCamera`
- `isHouseLocationFree`

- `processKeyboardInput`